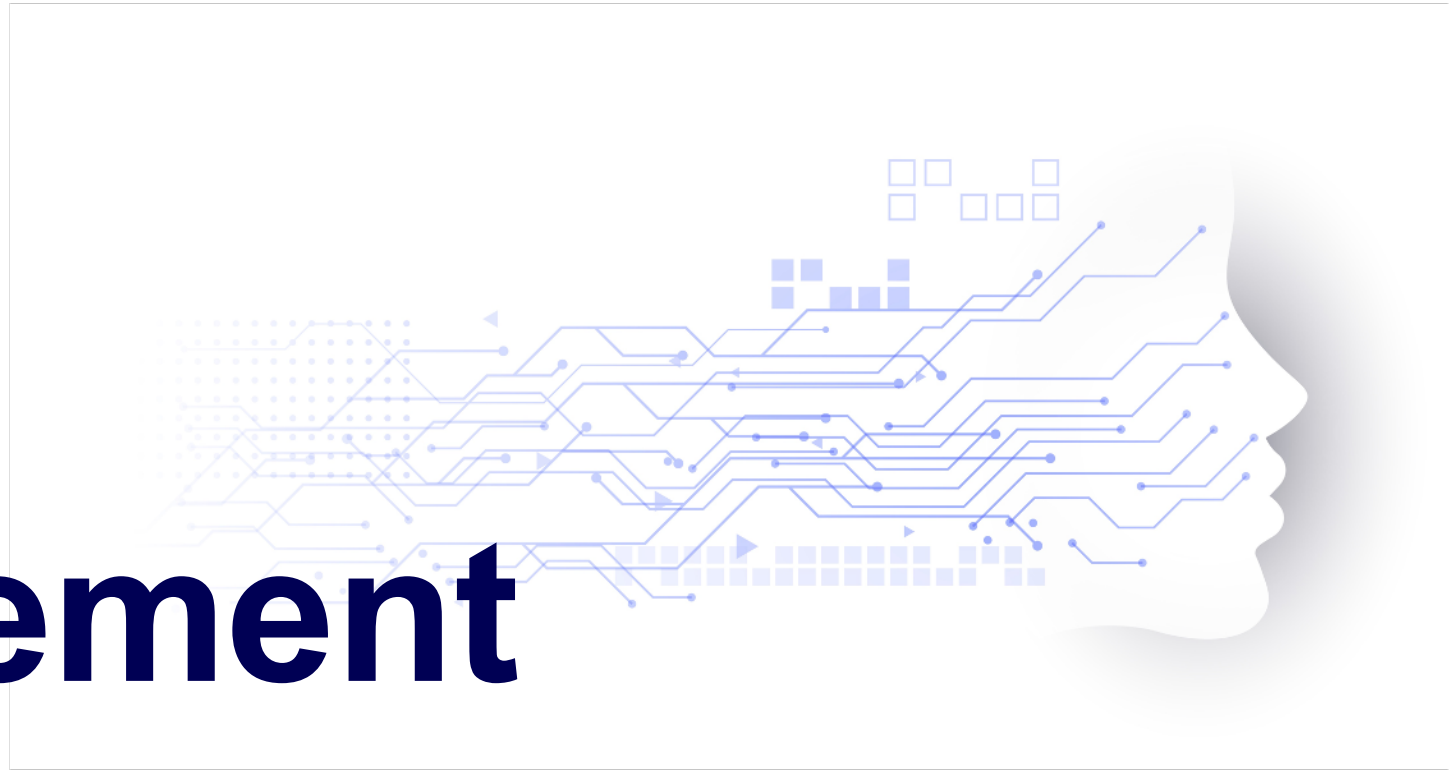
RMIT Classification: Trusted



Reinforcement Learning

COSC 2673-2793

(Computational) Machine Learning

Image: Freepik.com

RMIT UNIVERSITY

Agenda

- RL
  - Basics of RL
  - Model based RL
  - Model Free RL

Revision: Supervised Learning

In ***supervised learning***, the output is known:

$$y = f(\mathbf{x})$$

Experience: Examples of input-output pairs

Task: Learns a model that maps input to desired output
Predict the output for new "unseen" inputs.

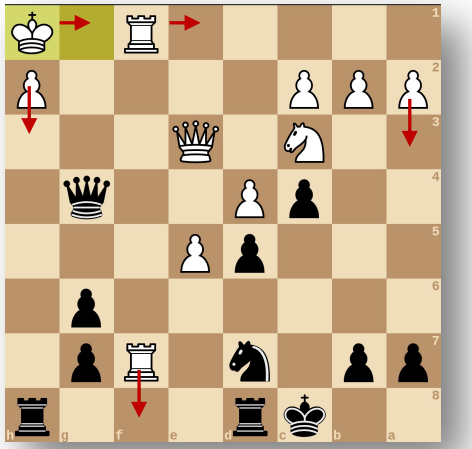
Performance: Error measure of how closely the hypothesis predicts the target output

Most typical of learning tasks

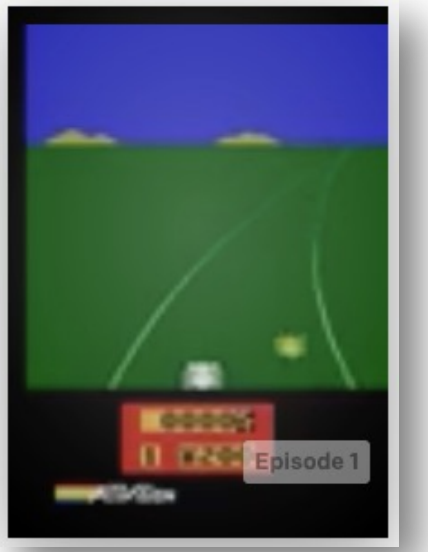
Two main types of supervised learning:

- Classification
- Regression

Reinforcement Learning



Learning to Play Chess



Learning to Play Racing Game



Learning to do Pick-and-Place

"What is the next "best" move?"

$$a = f(S)$$

Reinforcement Learning

Premise:

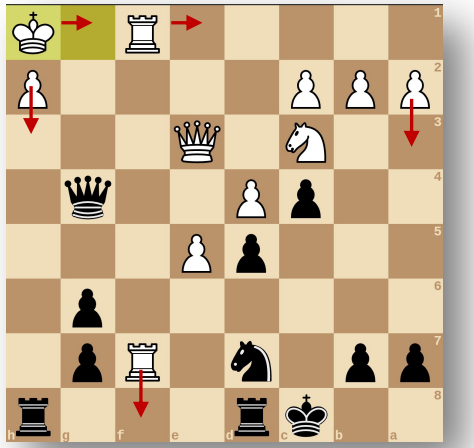
An ***Agent*** operating in an environment:

- The Agent can ***sense*** the world
- The Agent can interact with the world and execute ***actions***
- The Agent has a ***goal(s)*** to achieve or ***task(s)*** to accomplish
- The Agent is executing actions over time, making this a ***temporal*** problem



The Agent learns how to reach the goal or complete the task:

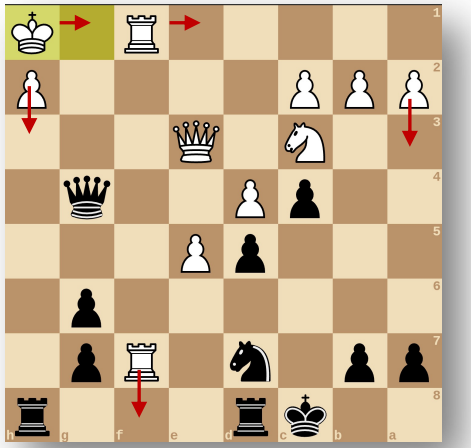
- The agent receives ***rewards*** that are relevant to the goal or task as it executes actions.
- From these rewards the agent must deduce what action to execute at any point of time



Reinforcement Learning

What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor,
- Only a reward signal Feedback is delayed,
- Not instantaneous Time really matters (sequential, non i.i.d data)
- Agent's actions affect the subsequent data it receives

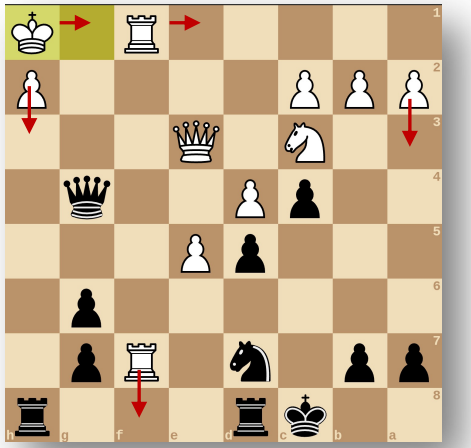




Reinforcement Learning

The ultimate purpose of the learning problem is for the agent to determine the ***optimal (best)*** action to execute at any point in time

This is equivalent for the agent determining the optimal action to execute in every state





RMIT Classification: Trusted

Basics of RL

6 Things to Know

1. State Space, S
2. Action Set, A
3. Transition Function, T
4. Reward Function, R
5. Value Function, V
6. Policy Function, π – similar to the target function

That is:

$$RL := \langle S, A, T, V, R, \pi \rangle$$

All forms of reinforcement learning are specific forms for, variations of, or extensions to, these 6 basic concepts.

State Space

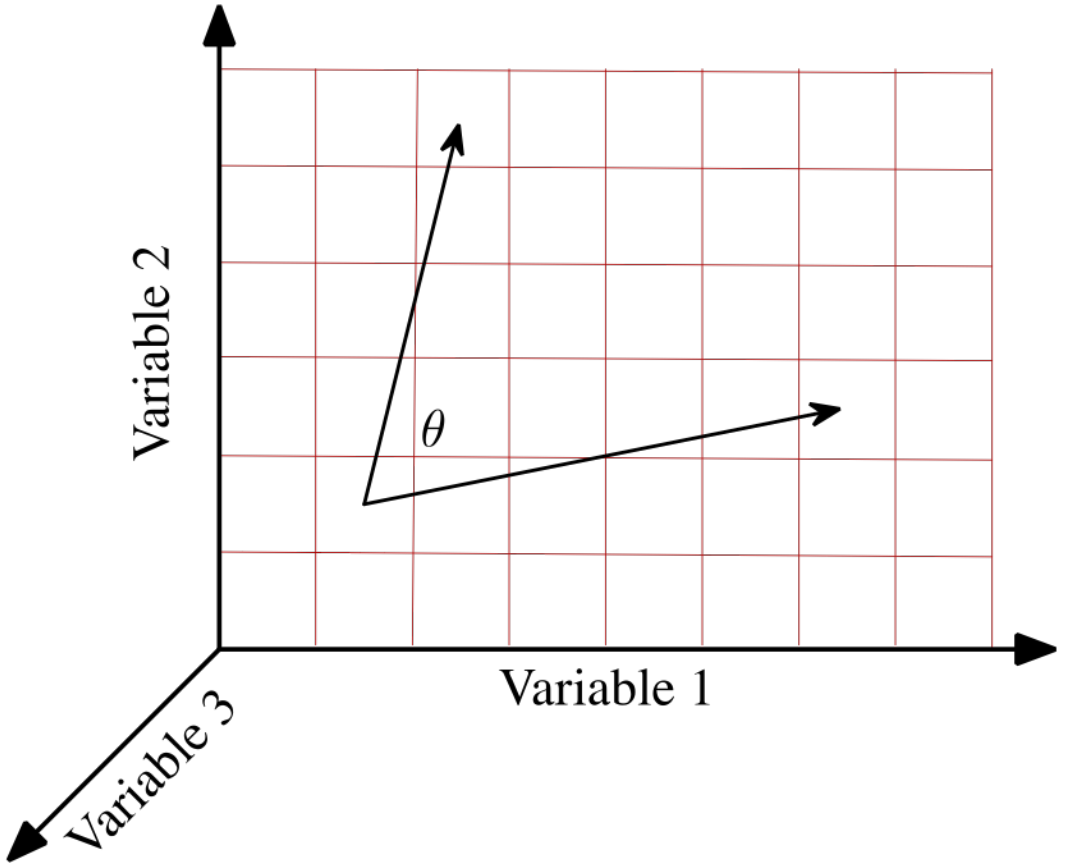
The ***state space***, S, is defined as the n-dimensional space formed from all valid combinations of the domains of every attribute:

$$S = x_0 \times x_1 \times \cdots \times x_n \forall x_i \in \mathbf{x}$$

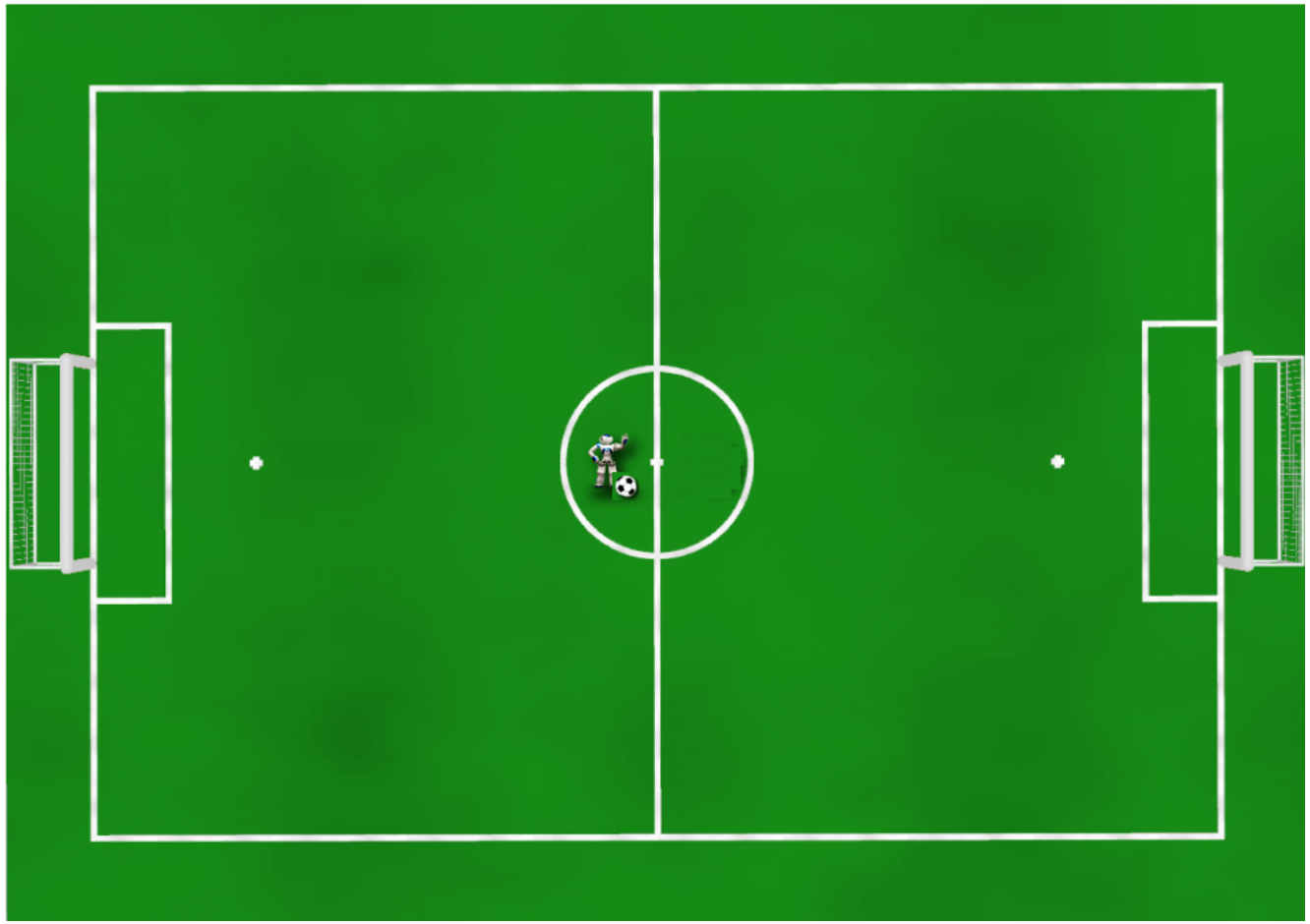
- The ***domain*** of an attribute is the set of all values that may be assigned to the attribute
- Based on the domain of each variable, the state-space may be: Infinite, bounded continuous but finite, discrete but continuous, partially ordered, unordered discrete/categorical, and more…

The agent is at/in one state, $s_i \in S$, within the state space at any point in time

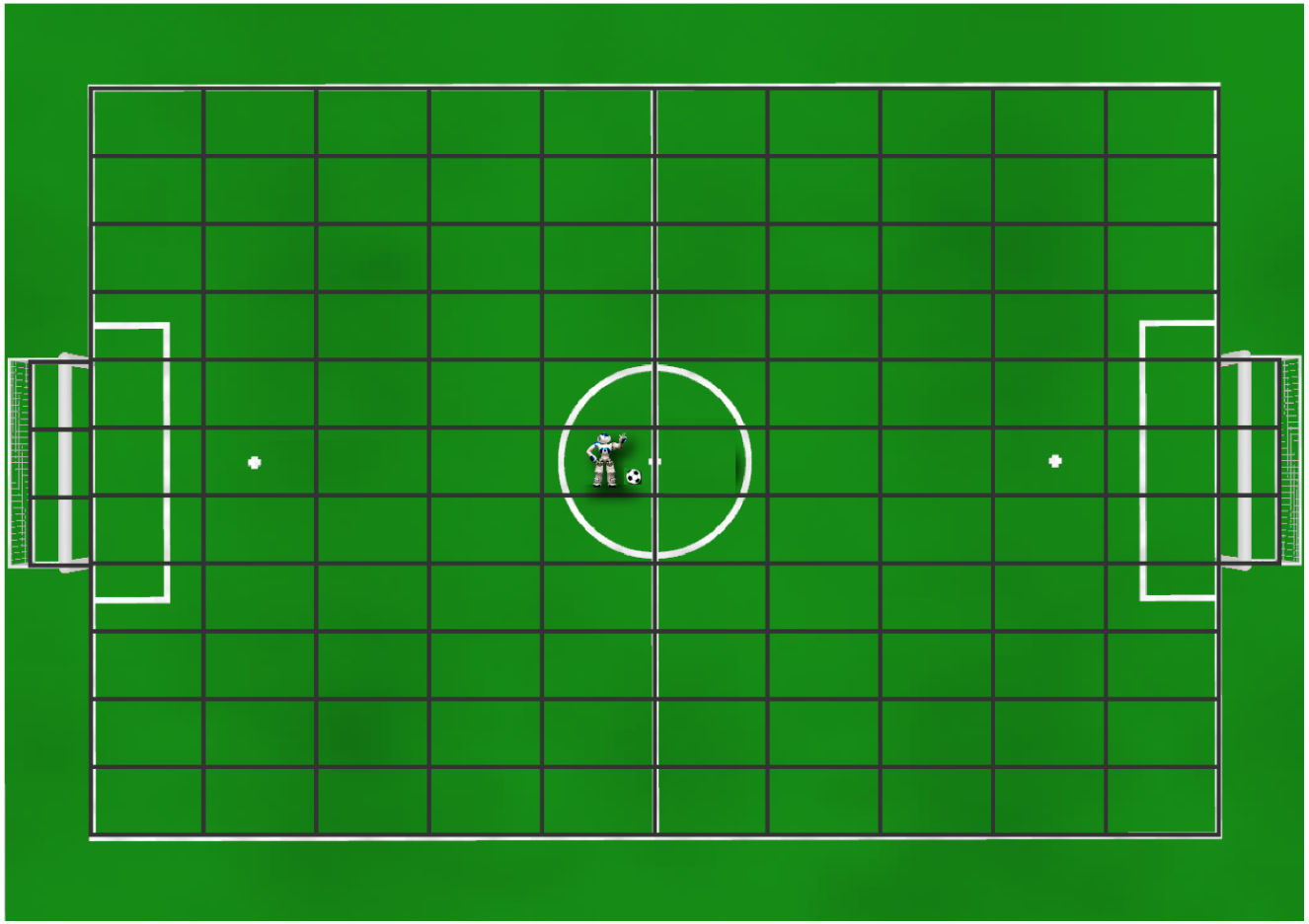
- As time progresses, the agent may remain in the same state, or the agent may change states



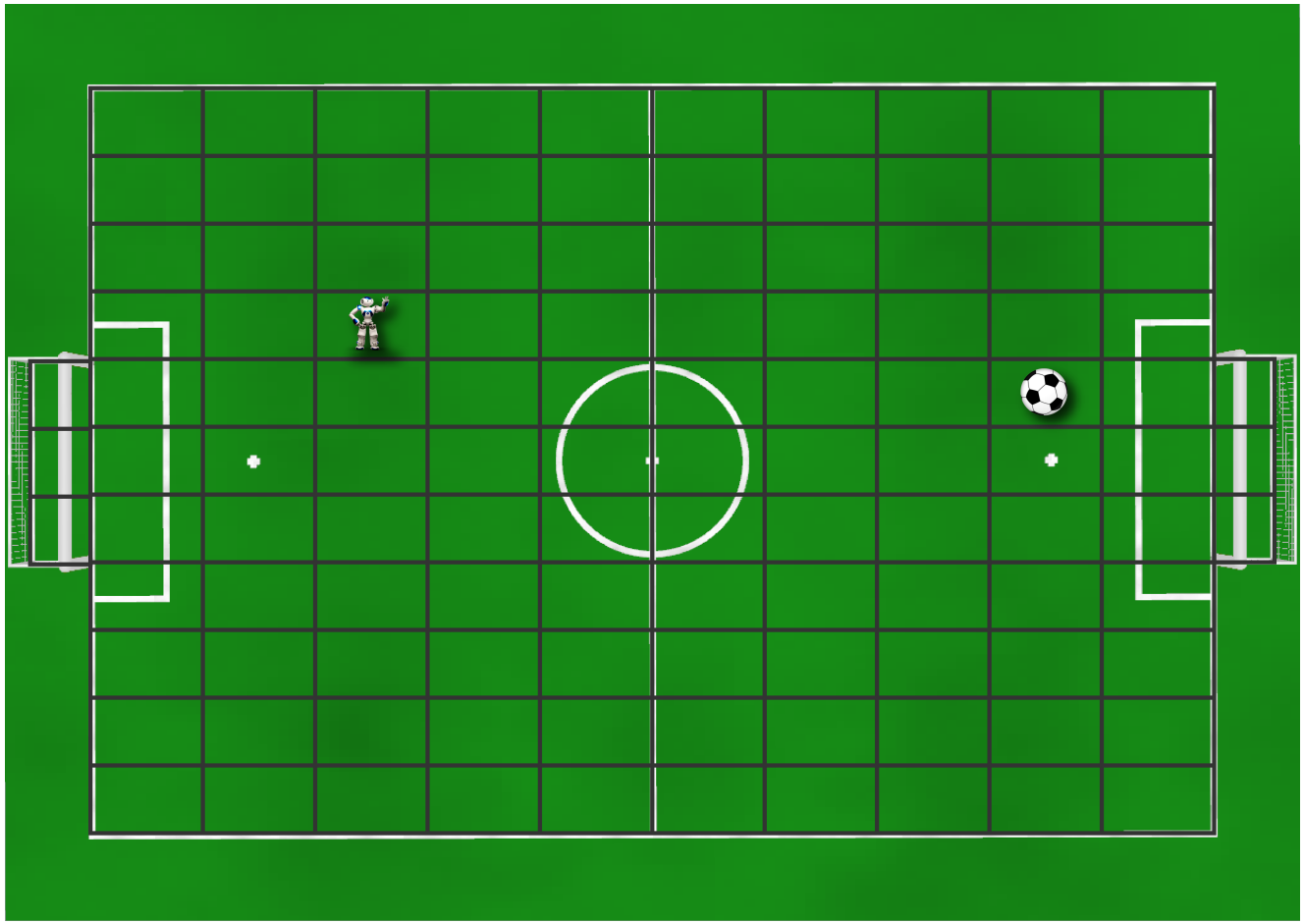
State Space: Soccer playing robot



State Space

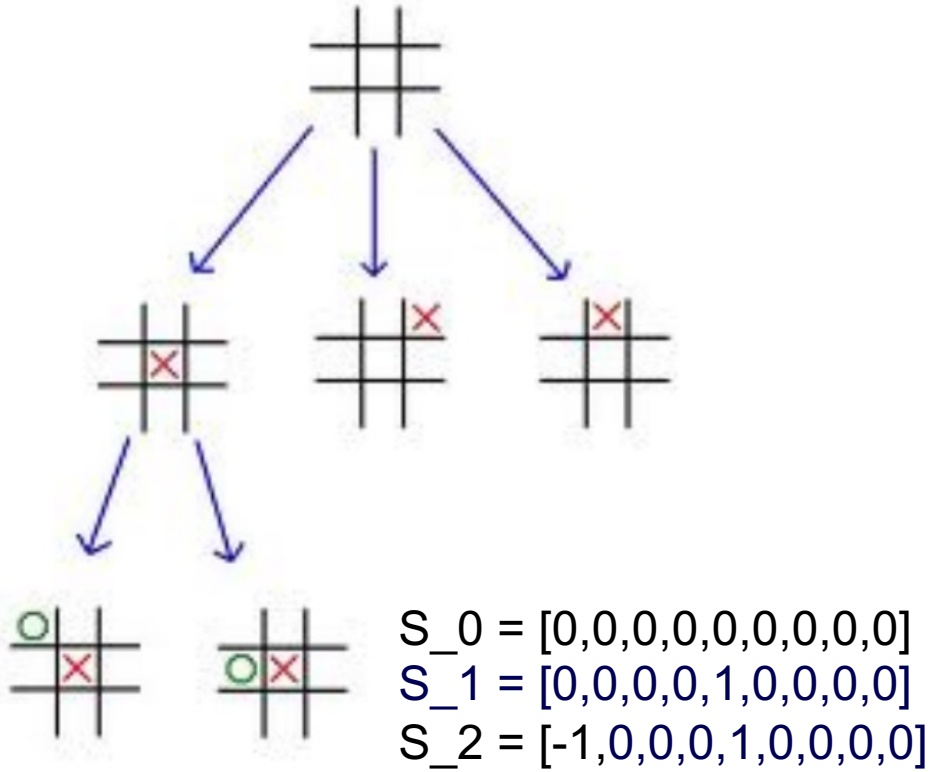


State Space



State Space





S_0 = [0,0,0,0,0,0,0,0,0]
S_1 = [0,0,0,0,1,0,0,0,0]
S_2 = [-1,0,0,0,1,0,0,0,0]

Action Set

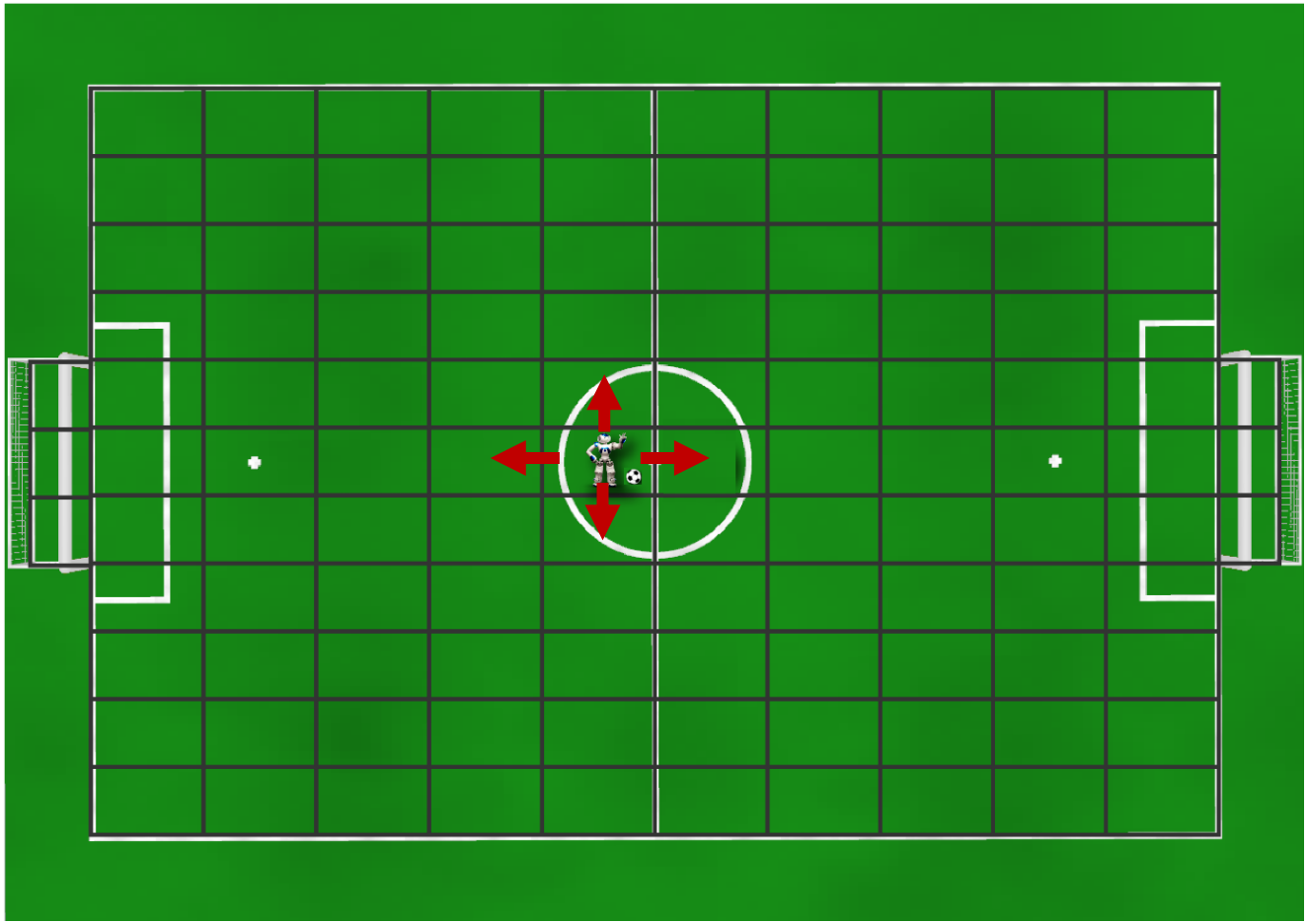
The ***Set of Actions***, A, are all of the actions that the agent may execute across the entirety of the state space.

- Actions correspond (or are mapped) to actuators of the agent
- It may not be possible for the agent to execute every action in every state – this is handled by the transition function. The set, however, must be complete
- For simple problems, the actions may be symbolic, but in complex tasks they may be conditional
- Actions are, generally, discrete choices

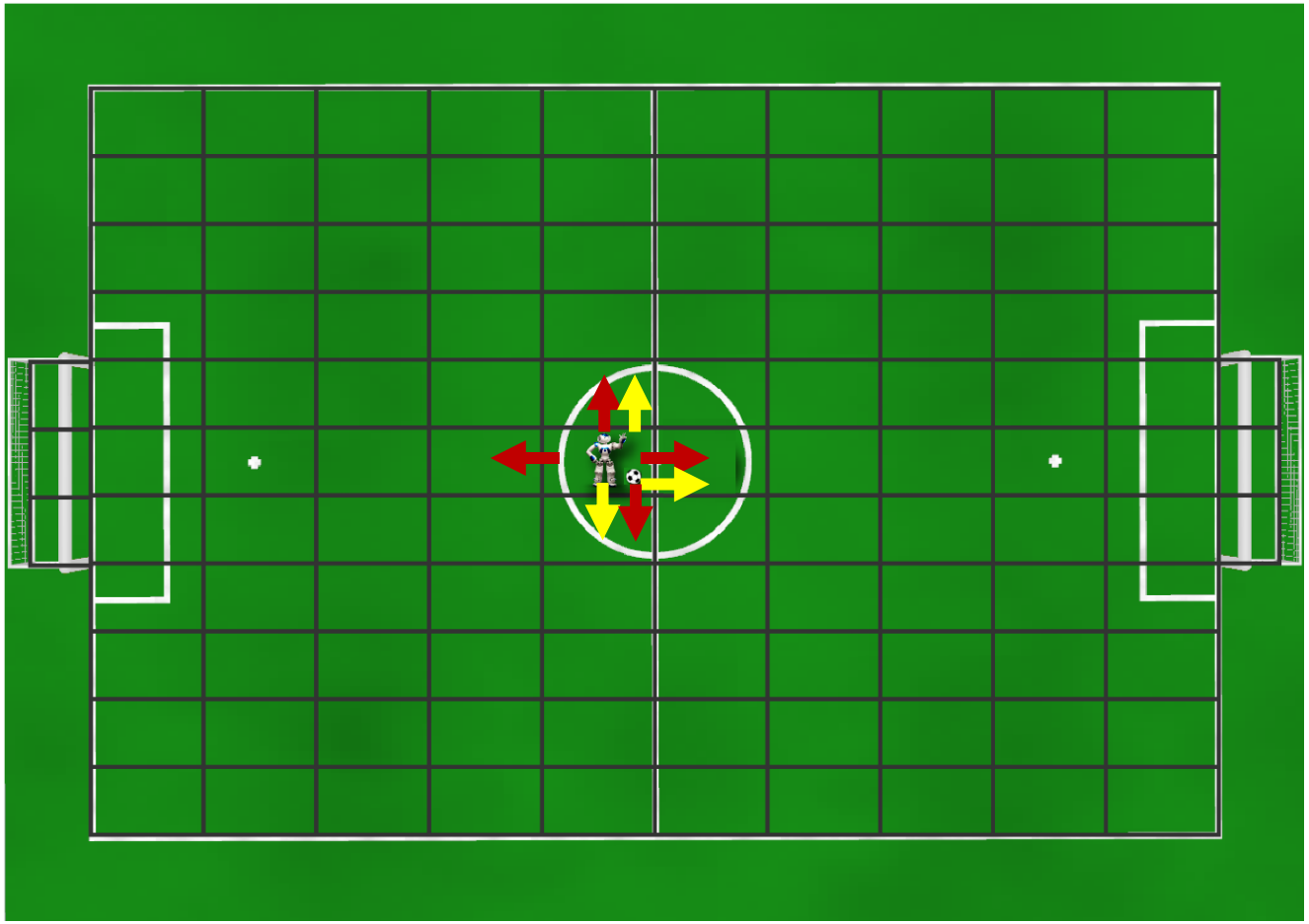
Importantly actions are NOT attributes of the state space.

- That is, $\forall a_i \in A, a_i \notin \mathbf{x}$

Action Set



Action Set

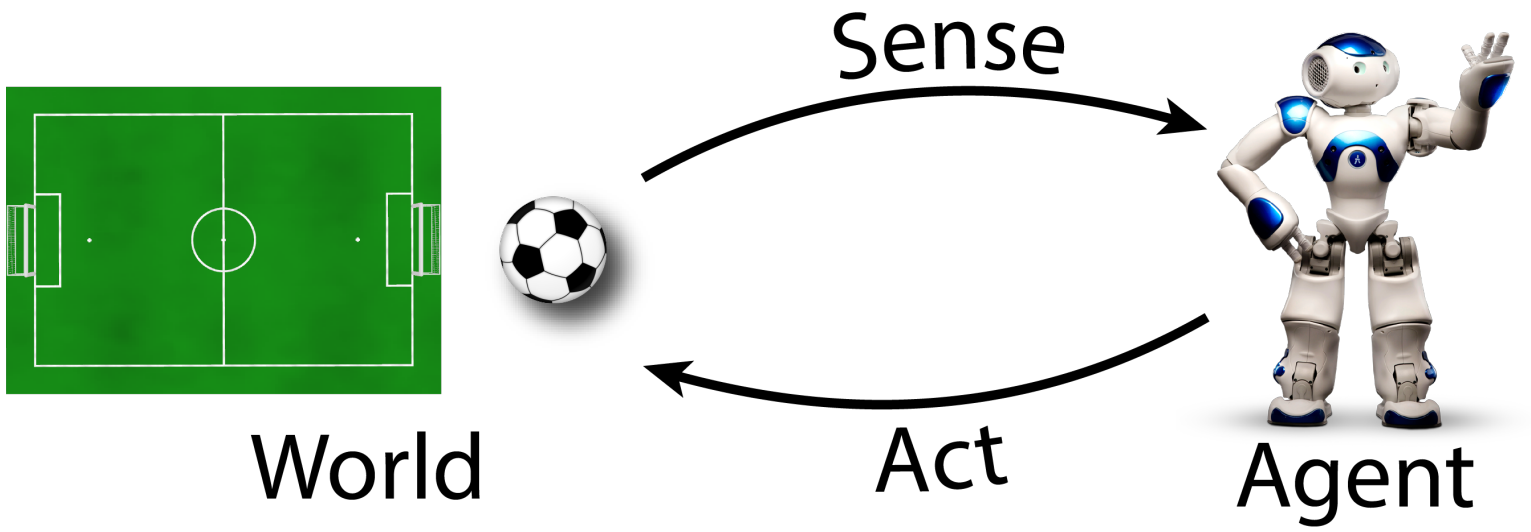




Sense-Act Cycle

For each time step, the agent:

- Senses the environment to determine it's state: $s_i \in S$
- Chooses an action: $a_j \in A$
- Executes the chosen action, within the duration of the time step



Transition Function

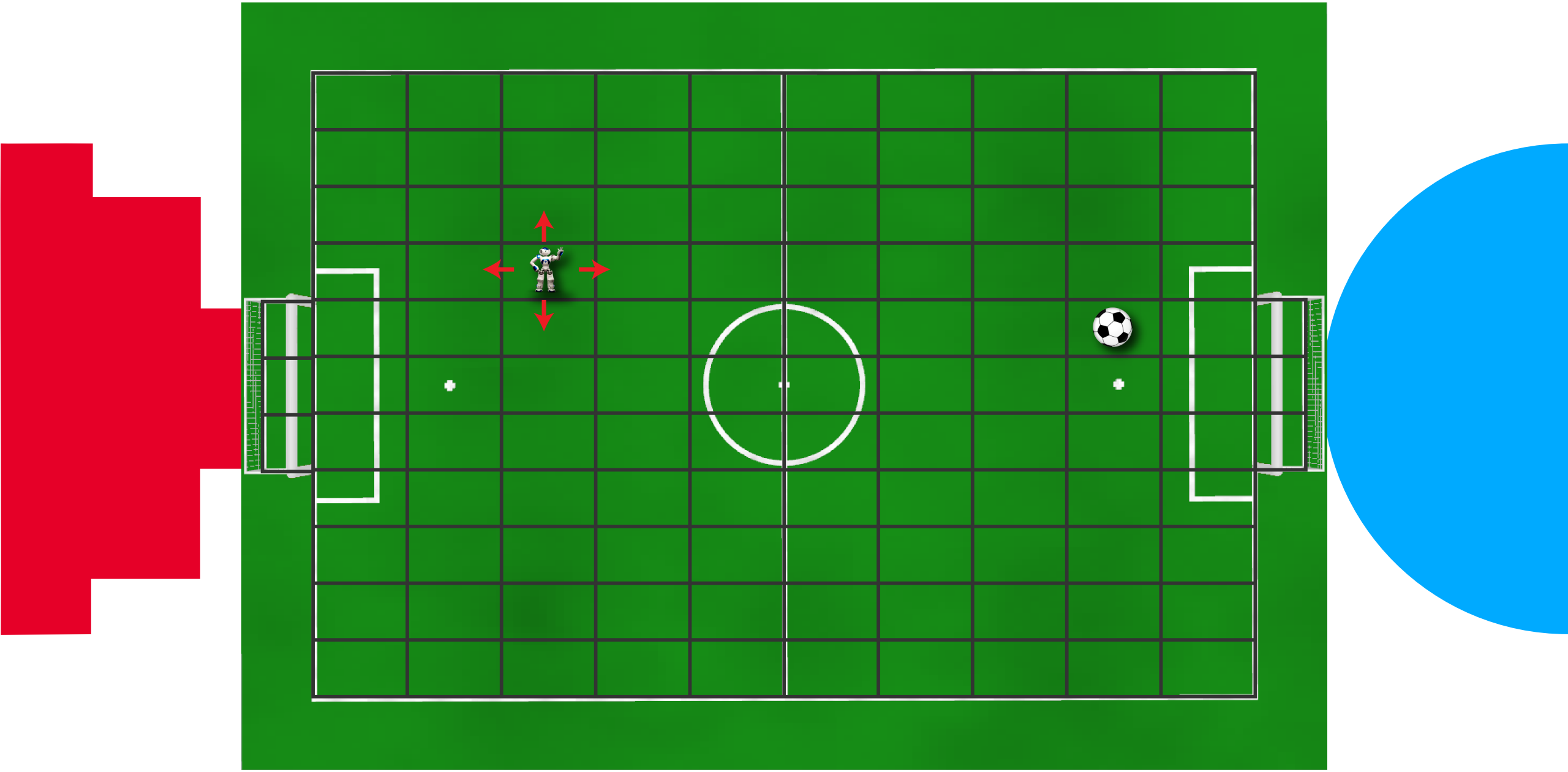
The ***transition function*** formally represents the process of the agent executing actions.

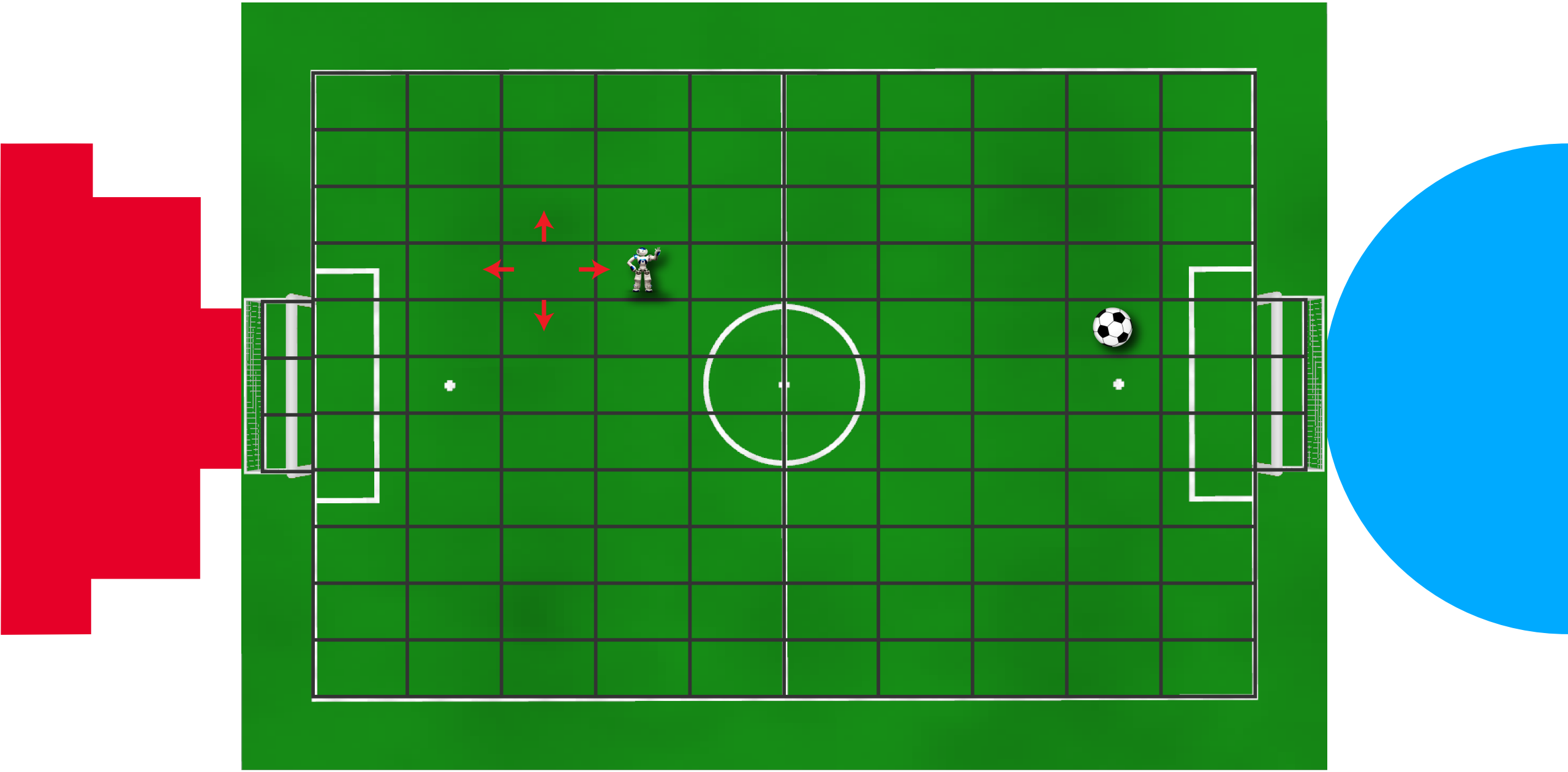
- The function maps state-action pairs to states
- The function computes the resulting state of the agent after executing an action in an initial state

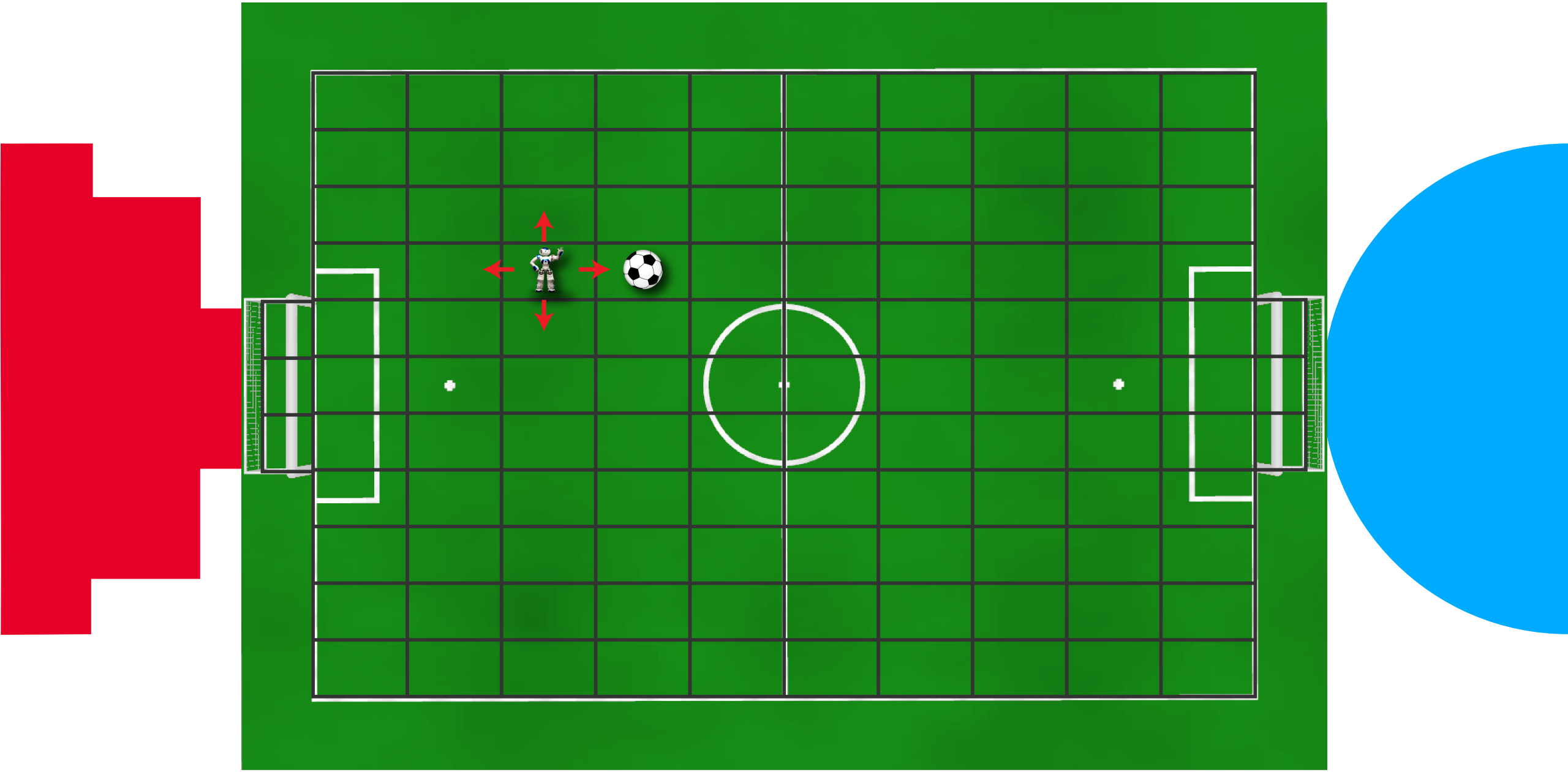
$$T := S \times A \rightarrow S$$

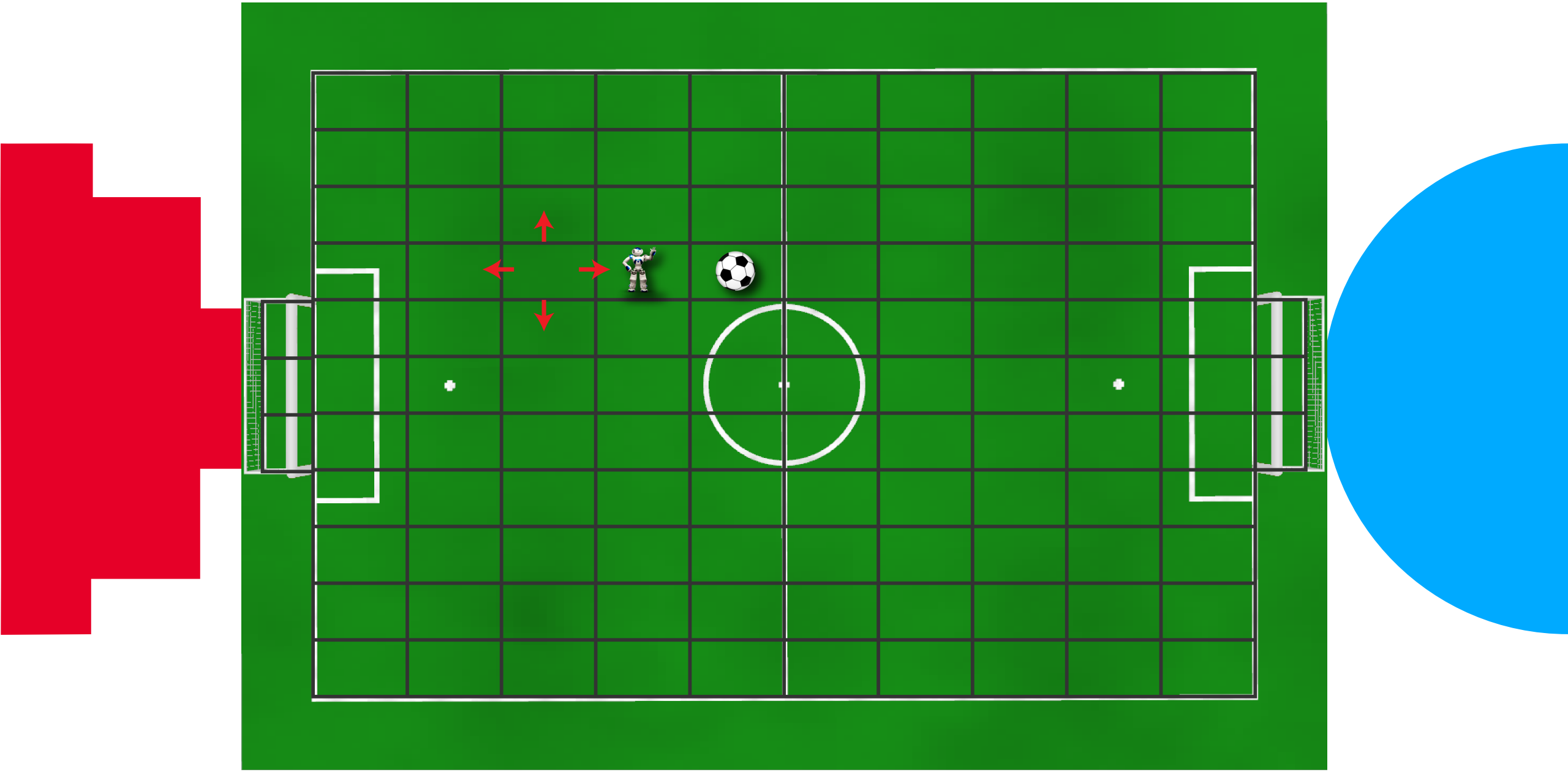
For each state-action pair

- If the agent may execute the action in the state, then a transition must be defined
- If the agent cannot action the state, then either:
  - The agent may remain in the same state, an an according transition is defined
  - No transition may be defined





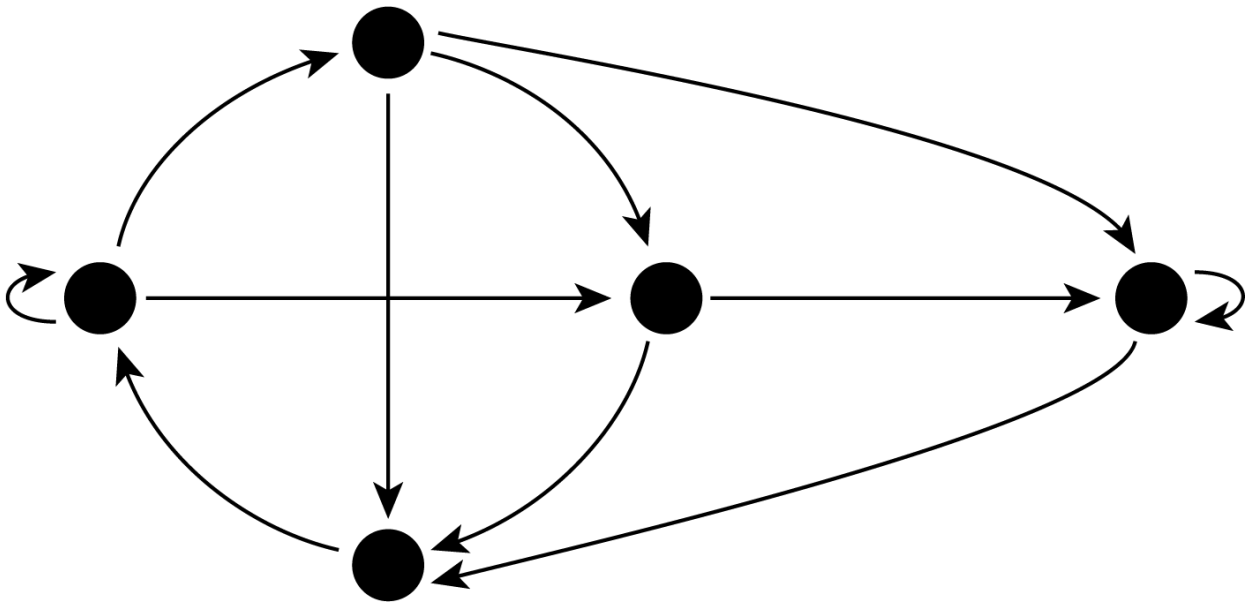




Transition Function

The transition function forms a directed graph of actions linking states

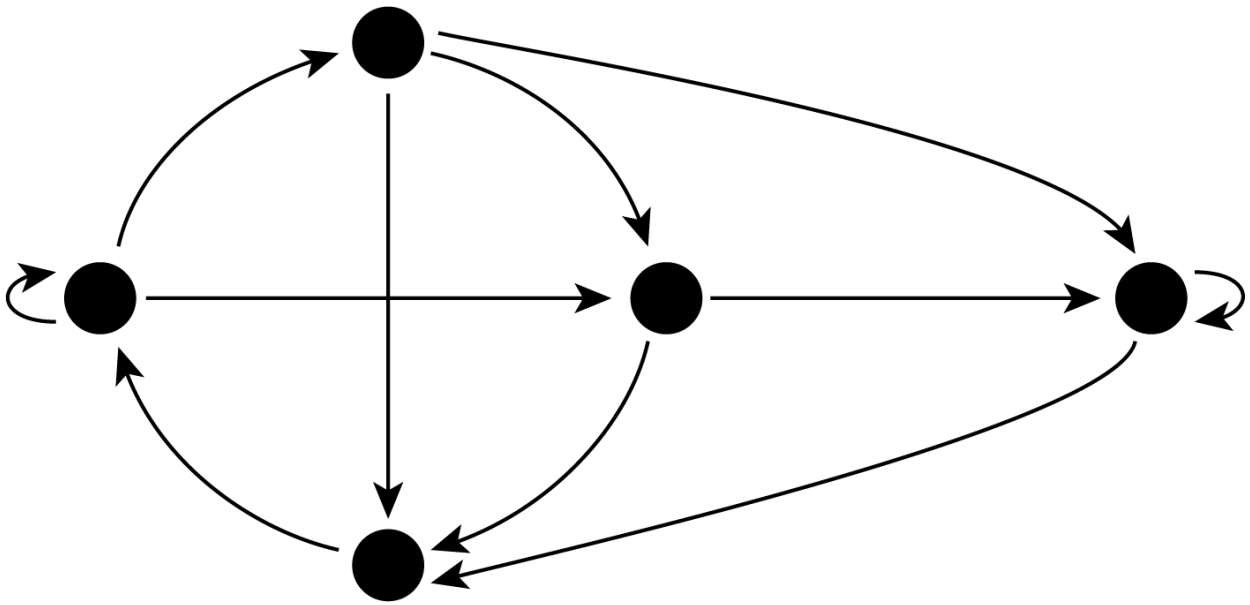
Note: this will become important later!



So how can Learning work?

What is the agent actually trying to learn?

For every state, what is the action that the agent should execute to achieve it's goal?



Reward Function

The ***reward function*** provides the reinforcement measure from which the agent is able to learn.

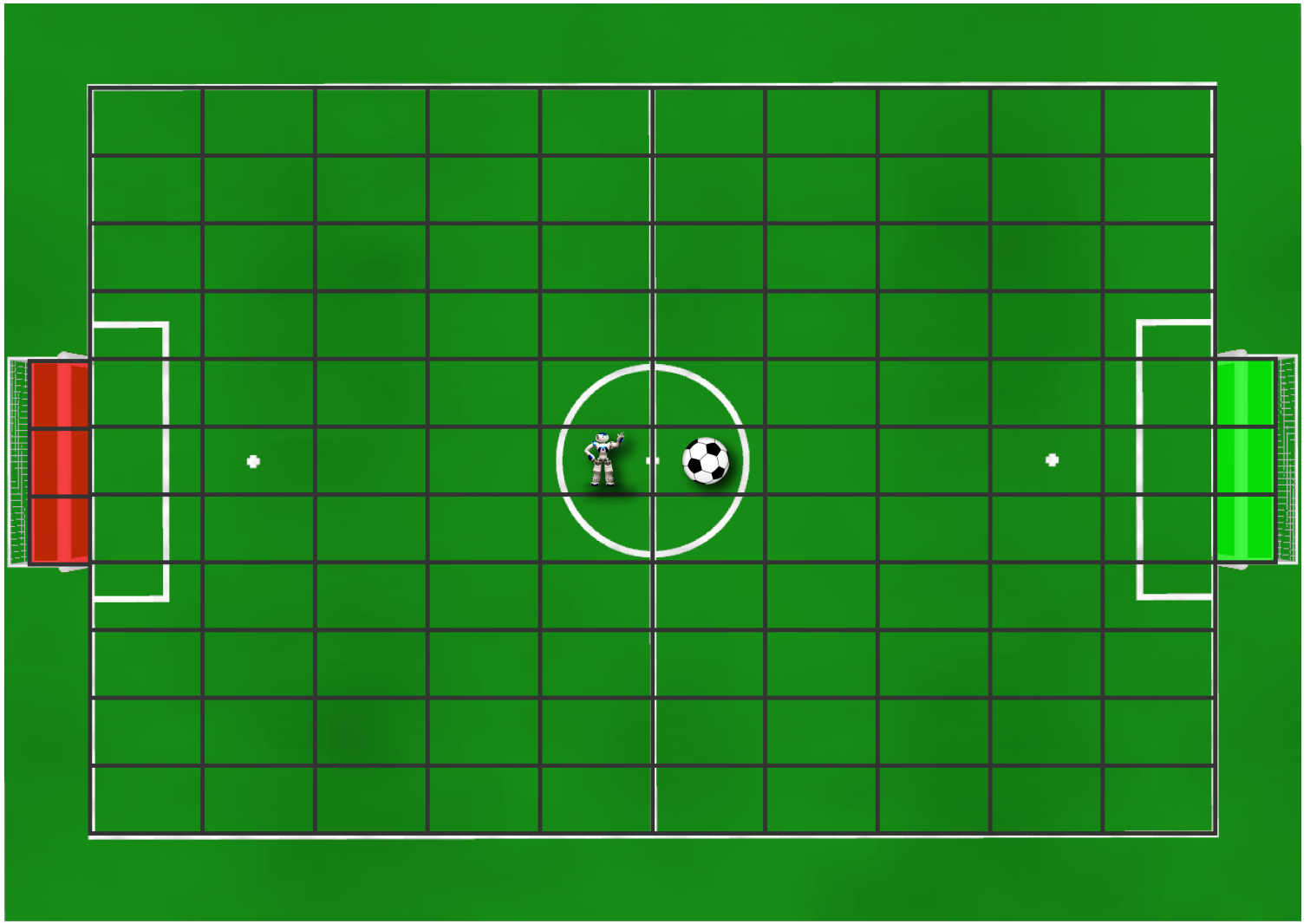
- The reward is typically tied to the states of the system, the actions of the agent, or both.
- Not every state or action may result in a reward, that is, the reward is zero.
- Rewards can be positive (the agent gains a benefit for reaching a state, or executing an action), or the rewards may be negative (the agent is "punished")

The reward function may be defined as:

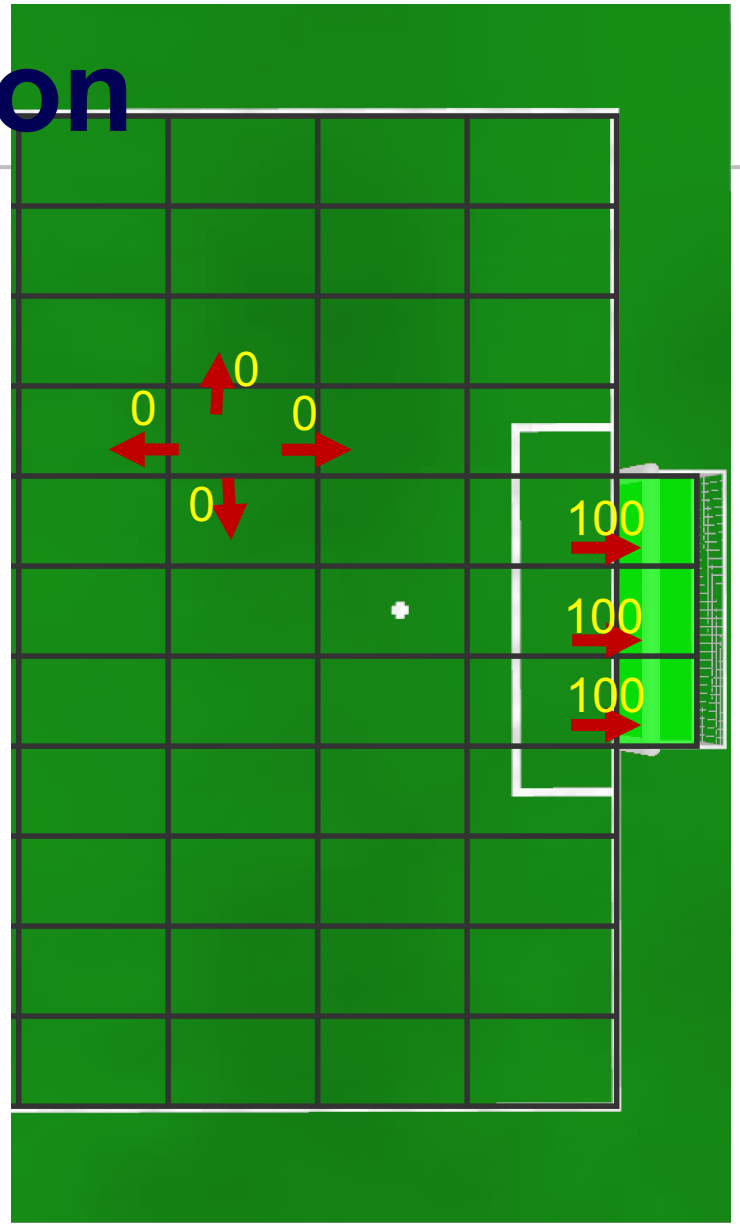
$$R := S \times A \rightarrow \mathbb{R}$$

- The reward is always a continuous real number
- The reward maps state-action pairs to rewards

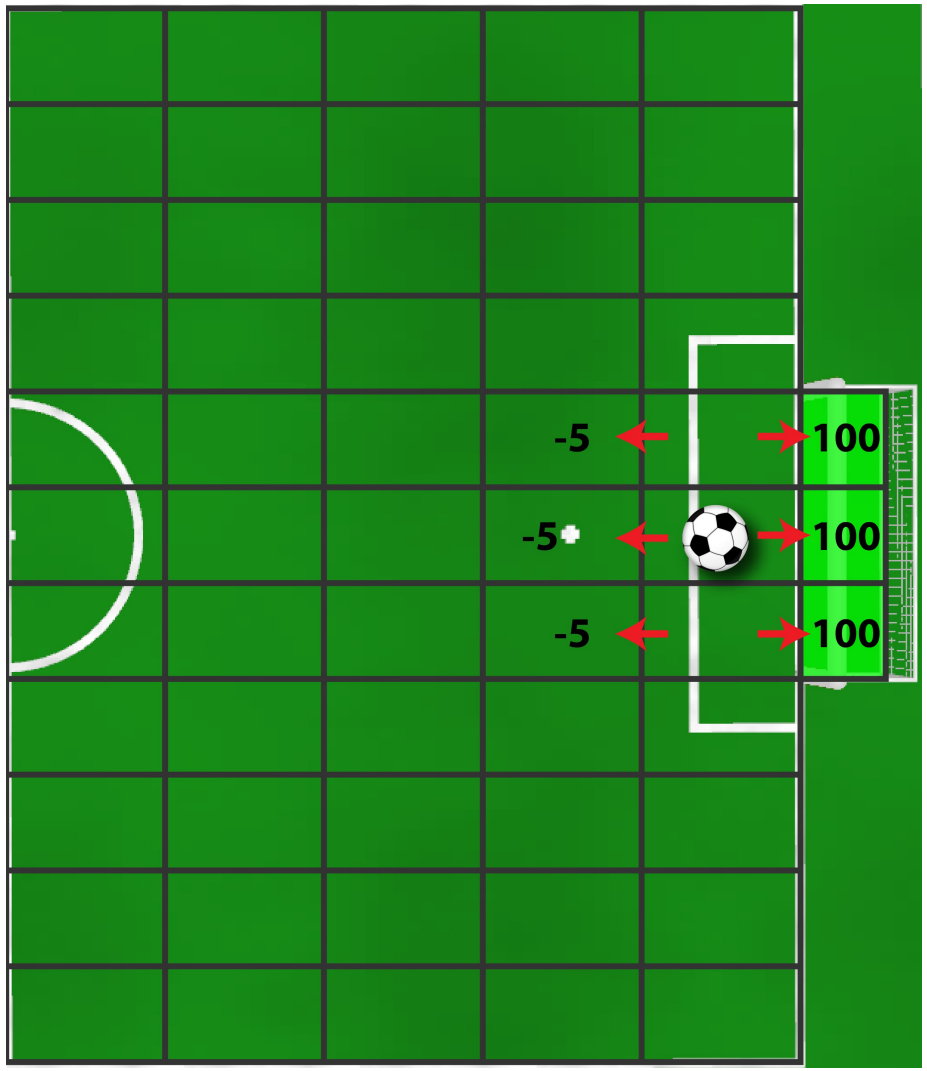
Reward Function



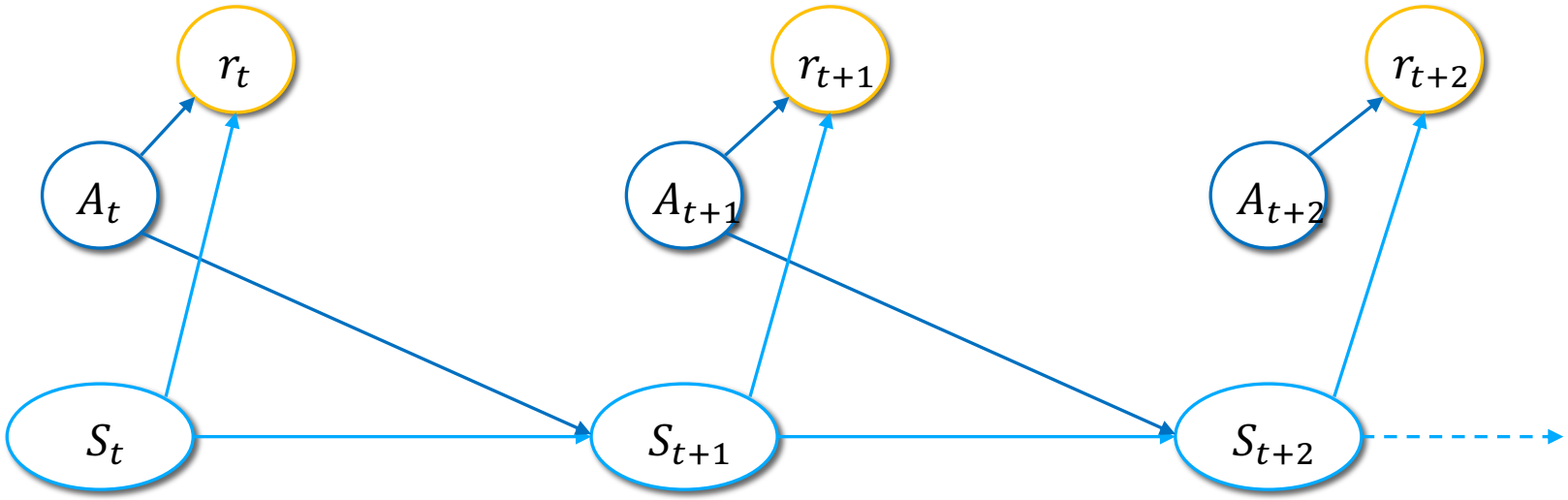
Reward Function



Reward Function



Task



Learn action that maximizes:

$$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots$$

$$E\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots\right] = E\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

Markov Decision Process (MDP)

A ***Markov Decision Process (MDP)*** is a decision problem where:

The decision only requires information that is available in the current state

$$P(S_{t+1} \mid S_t, a_t, S_{t-1}, A_{t-1}, \cdots) = P(S_{t+1} \mid S_t, a_t)$$

$$P(r_{t+1} \mid S_t, a_t, S_{t-1}, A_{t-1}, \cdots) = P(r_{t+1} \mid S_t, a_t)$$

The history of states and decisions that the agent took in reaching it's current state has no impact on the decision

It is critical that a reinforcement learning problem is properly defined as an MDP.

Policy Function

The ***policy function*** provides the action to choose in any state:

$$\pi := S \rightarrow A$$

The ***optimal policy*** is the best action to execute in any state, denoted by:

$$\pi^*$$

Policy Function

But how do you define the policy? How do you choose the best action to execute?

Use the transition function!

From the agent's current state:

- Look at all the possible actions
- Choose the action that leads to the best state
- What is the best state? This is given by the value function

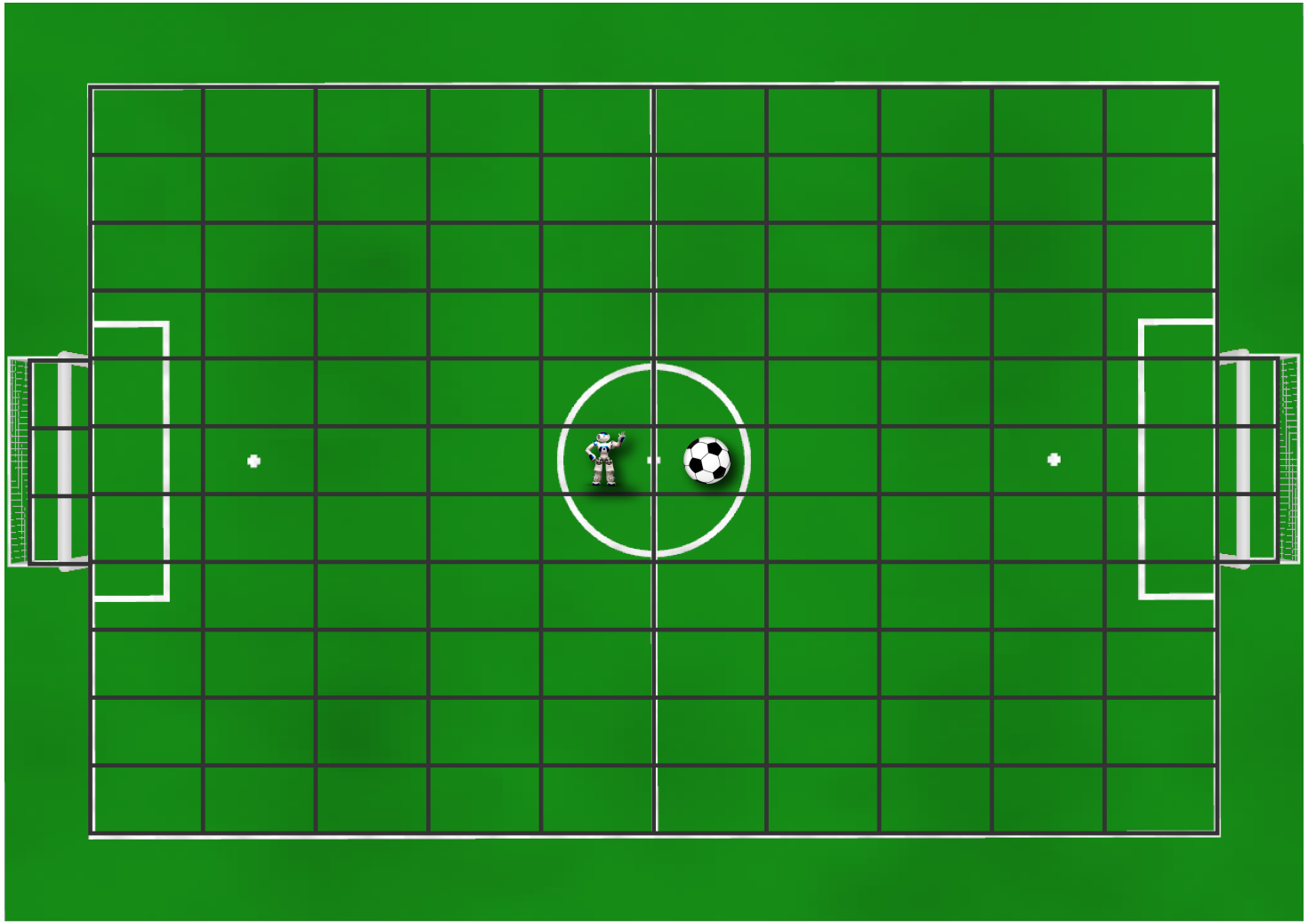
But!, the devil is in the detail.

This makes a very significant assumption:

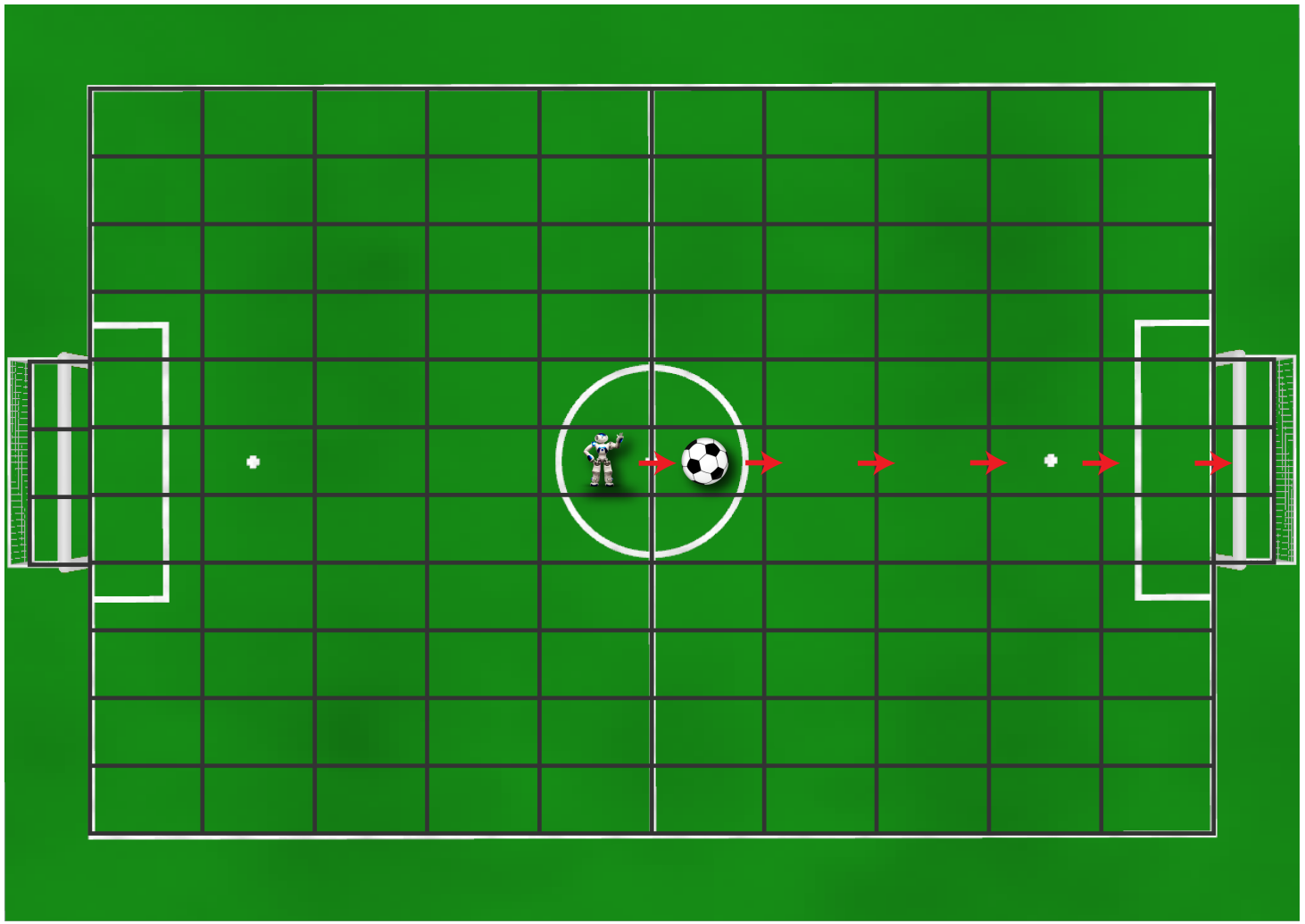
The choice of the best action a can be made by ONLY considering the current state of the agent s

Reinforcement learning is a form of a ***Markov Decision Process***

Policy Function



Policy Function



Value Function

The ***value function*** is the measure of the "worth" or "quality" of each state:

$$V := S \rightarrow \mathbb{R}$$

How do we actually measure/define the "quality" of a state?

- As the agent get's closer to the goal the value of states should increase

Value Function

The value is proportional to:

The ***immediate reward*** that the agent receives upon leaving the state

The ***expectation of future rewards*** the agent will receive by after executing future actions

That, is:

$$V(S) = E\left[r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \cdots\right] = E\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

- The value of state at timestep i is the sum of the immediate reward r_i and future rewards looking ahead t timesteps (or t actions)

Value Function

$$V(S) = E\left[r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \cdots\right] = E\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

$$V^*(s) = E\left[R(s, \pi^*(s))\right] + \gamma E\left[V^*(s')\right]$$

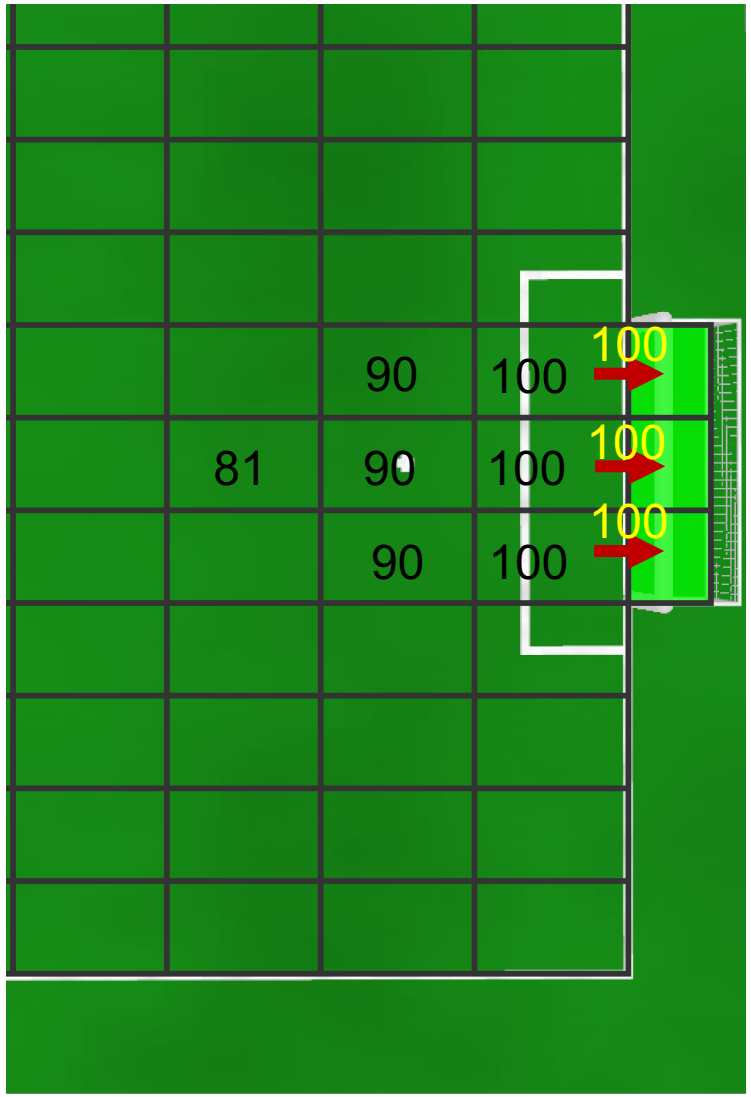
With the reward function, we can devise a form for the value function

$$V(s) = \max_a [R(s,a) + \gamma V(s')]$$

Where $s' = T(s,a)$

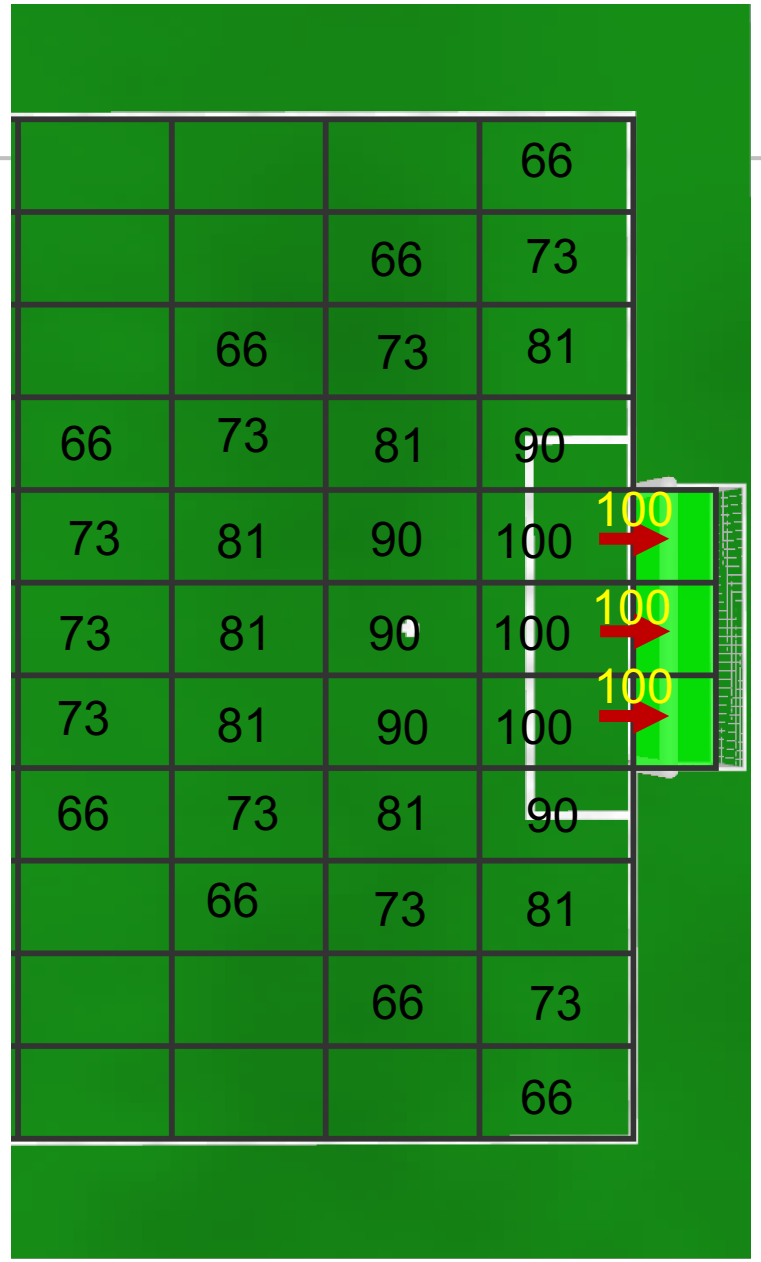
Value Function

$\gamma = 0.9$



Value Function

$\gamma = 0.9$



Policy Function

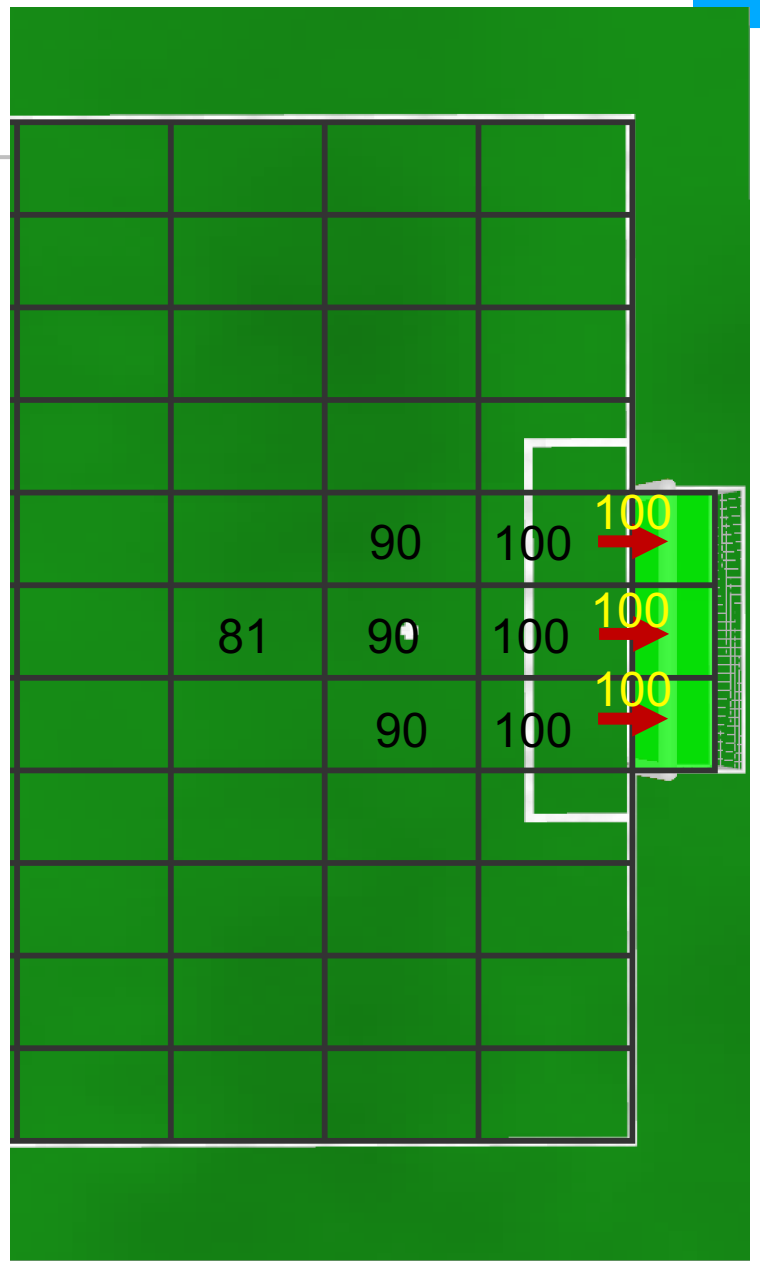
Thus policy function can also be calculated:

$$\pi(s) = \underset{a}{\text{argmax}}[R(s,a) + \gamma V(s')]$$

Where $s' = T(s,a)$

However, the policy is optimal, if and only if, the value function gives an optimal value

$$\pi^*(s) = ?$$



Policy Function

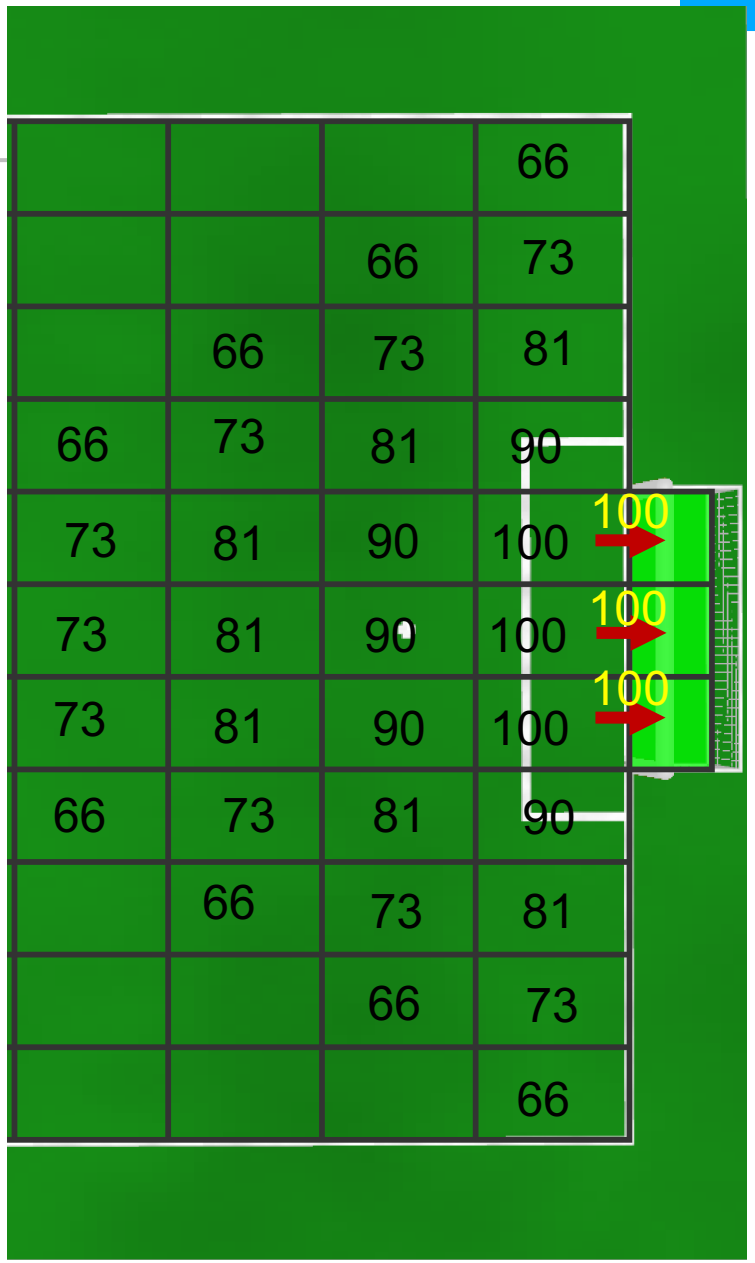
Thus policy function can also be calculated:

$$\pi(s) = \underset{a}{\text{argmax}}[R(s,a) + \gamma V(s')]$$

Where $s' = T(s,a)$

However, the policy is optimal, if and only if, the value function gives an optimal value

$$\pi^*(s) = ?$$



6 Things to Know

1. State Space, S
2. Action Set, A
3. Transition Function, $T := S \times A \rightarrow S$
4. Reward Function, $R := S \times A \rightarrow \mathbb{R}$
5. Value Function, $V := S \rightarrow \mathbb{R}$
6. Policy Function, $\pi : S \rightarrow A$

Computing the functions

So far we have only provided definitions for the key elements of RL problems.

But, where is the actual “learning”?

If the **optimal policy function is known**, then nothing more needs to be done.

Likewise, if the **value function is known**, then nothing more needs to be done.

We previously noted the policy function is optimal, if and only if, the value function is optimal

What is the value function isn’t known?

Then the value function needs to be computed from the reward and transition functions

- This is the actual reinforcement learning!

This gives two circumstances:

- ***Model-based RL***, where the reward and **transition functions are known**
- ***Model-free RL***, where the reward and **transition functions are unknown**

RMIT Classification: Trusted

Model-Based RL

- Value Iteration
- Policy Iteration

Model-Based RL

For ***Model-Based RL***, the reward and transition functions are fully defined (or known)

That is, for every state-action pair:

- The state the agent transitions into after executing the given action from the given state is known
- The immediate reward of the agent executing the given action from the given state is known

It is termed model-based since the “model” of the agent and system, is known

The two common model-based algorithms for learning the value and policy functions are:

Value Iteration

Policy Iteration

Value Iteration

As the name suggestion, ***value iteration*** repeatedly updates the value function until it converges

The iteration is necessary as the value function is recursive

Provided that the reward and transition functions are well-defined, that is Markovian, the value function will correctly converge to the optimal value

Value_Iteration($\langle S, A, T, R, V, \pi \rangle, \gamma$)

forall $s \in S$ initialise $V(s) = 0$

repeat until converges (does not change):

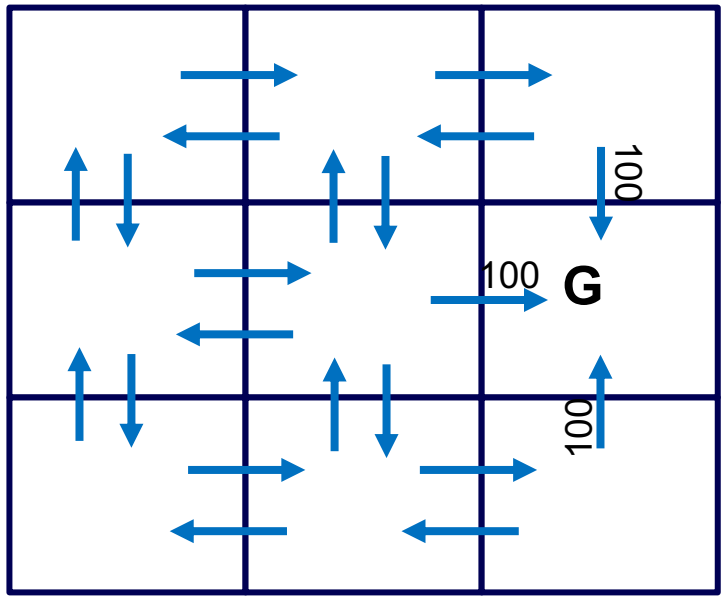
forall $s \in S, V(s)$

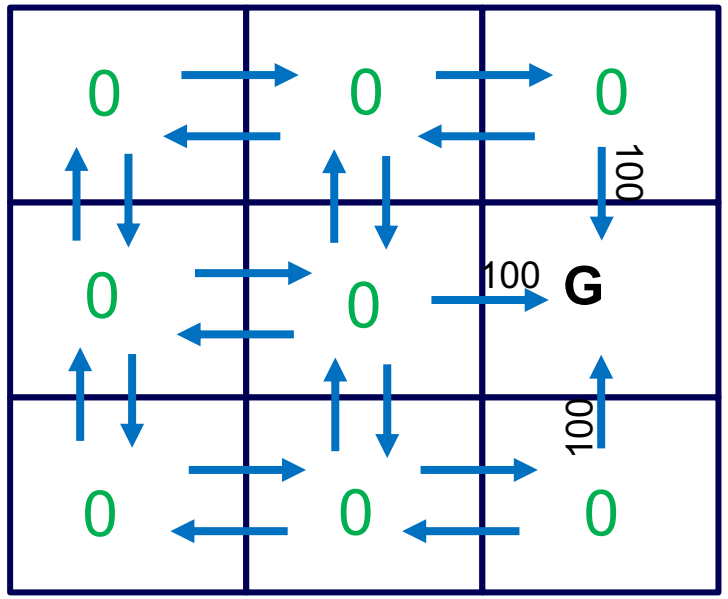
$$V(s) = \max_a [R(s,a) + \gamma V(T(s,a))]$$

forall $s \in S$ calculate:

$$\pi^*(s) = \operatorname{argmax}_a [R(s,a) + \gamma V(T(s,a))]$$

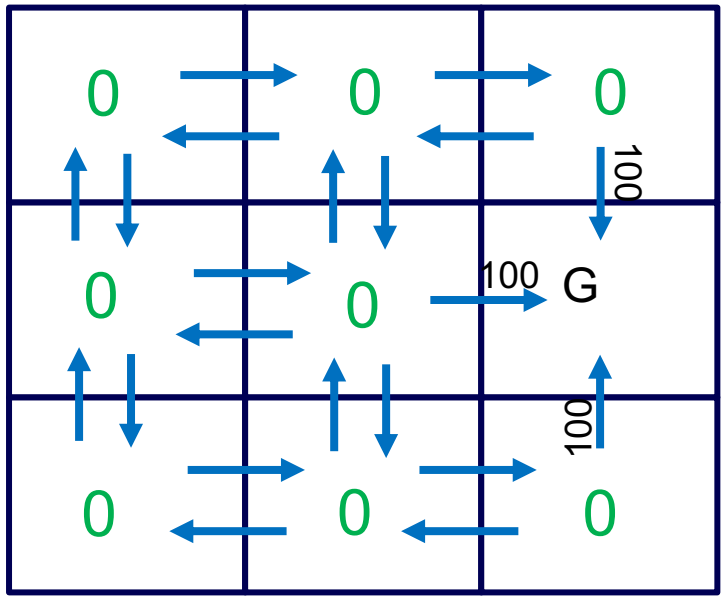
Value Iteration

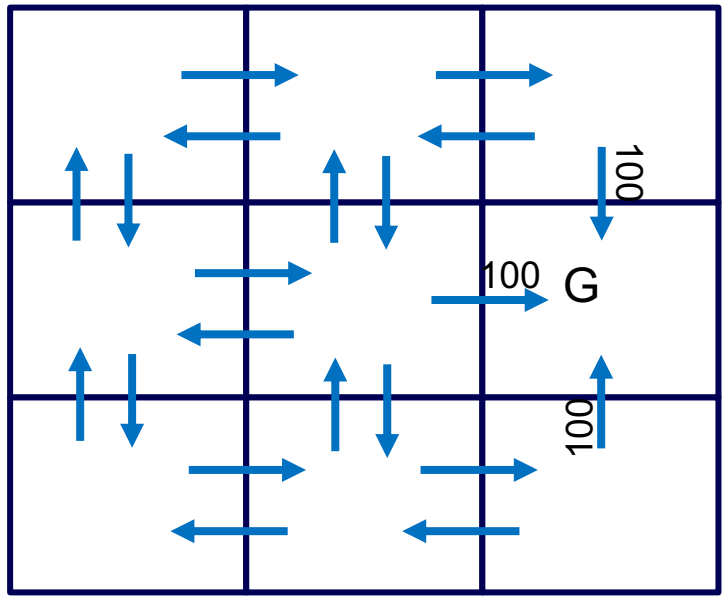




$\gamma = 0.9$

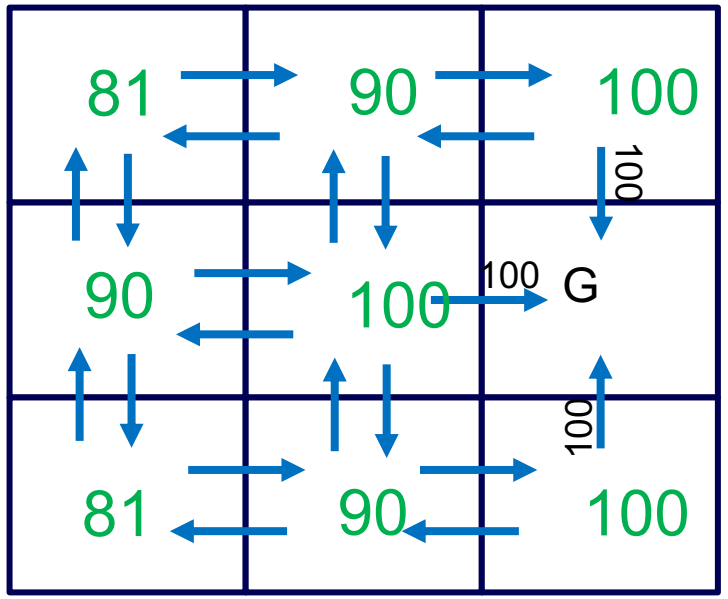
Value Iteration



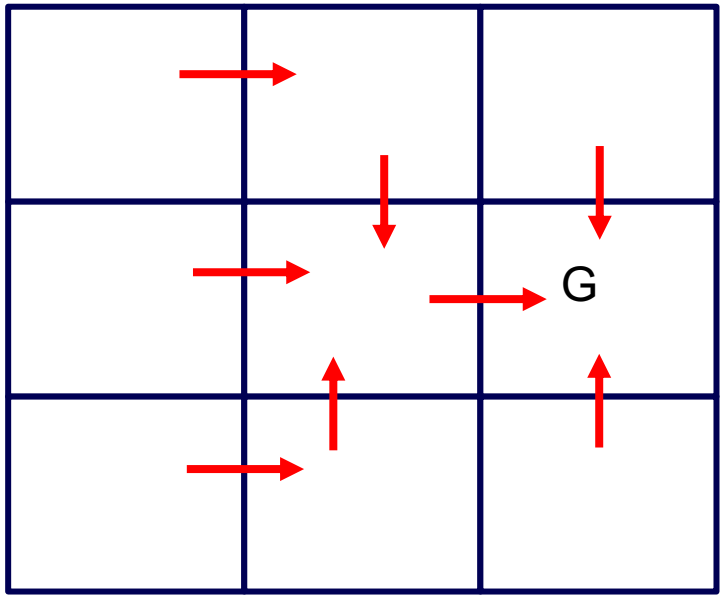


$\gamma = 0.9$

Value Iteration



$\gamma = 0.9$ V^*



π^*

Policy Iteration

Similarly, ***policy iteration*** repeatedly updates the policy function until it converges

In each loop, the value function is update according to the policy, and then the policy is re-computed

PolicyIteration($\langle S, A, T, R, V, \pi \rangle$,$\gamma$(

forall $s \in S$ initialise $\pi(s)$ randomly

repeat until $\pi(s)$ converges:

forall $s \in S$, update until $V(s)$ converges:

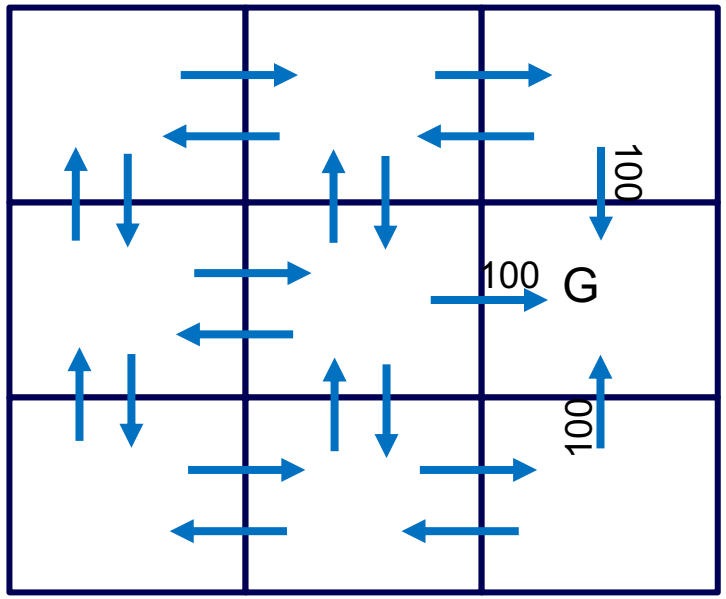
$$V(s) = R(s, \pi(s)) + \gamma V\big(T(s, \pi(s))\big)$$

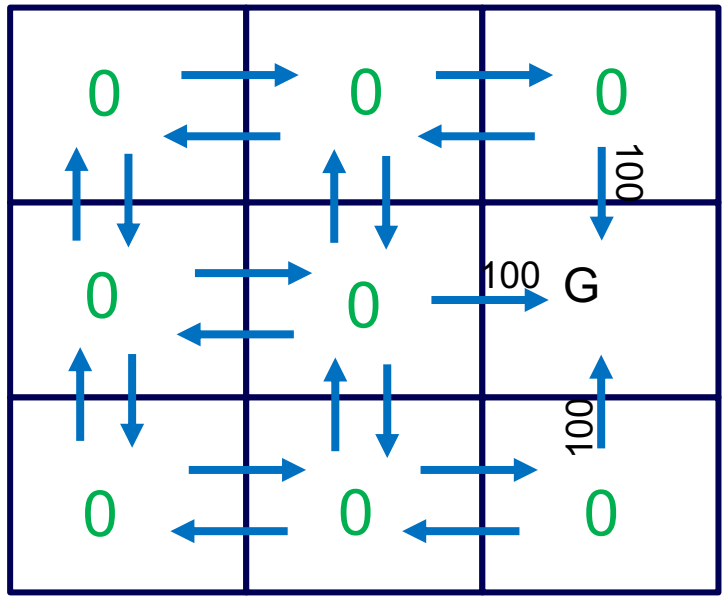
forall $s \in S$ calculate:

$$\pi(s) = \underset{a}{\operatorname{argmax}}[R(s, a) + \gamma V\big(T(s, a)\big)]$$

return $\pi(s)$ as $\pi^*(s)$ forall $s \in S$

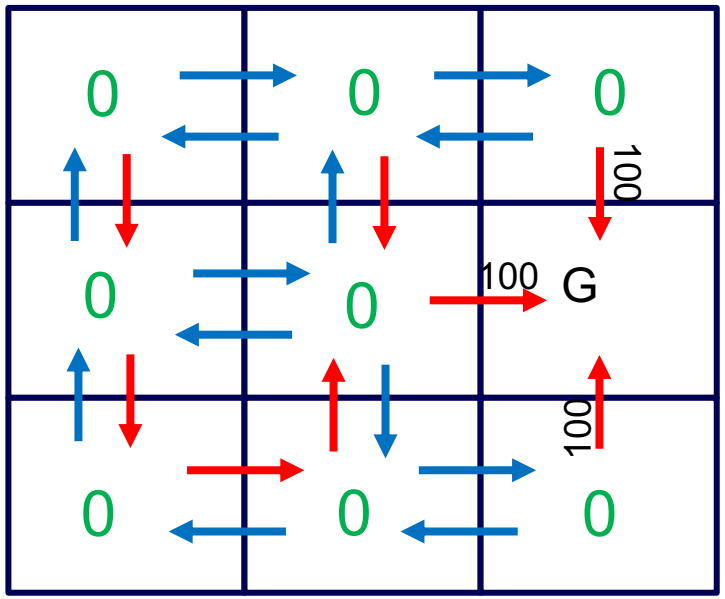
Policy Iteration

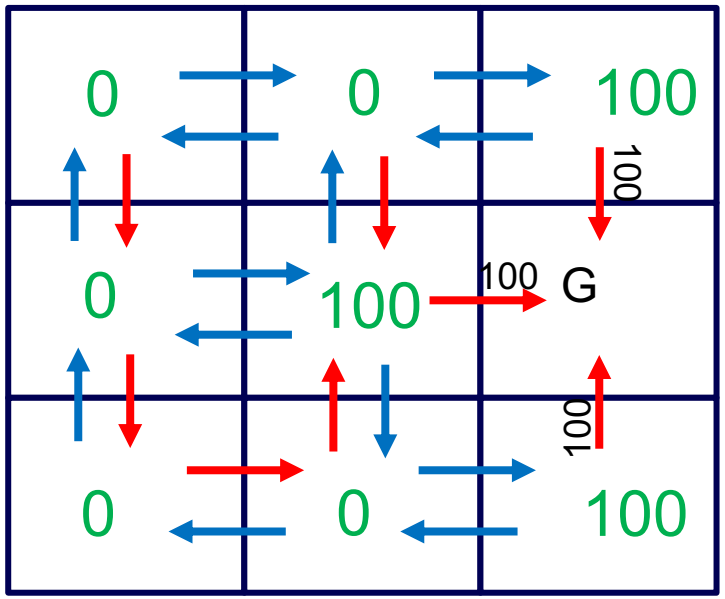




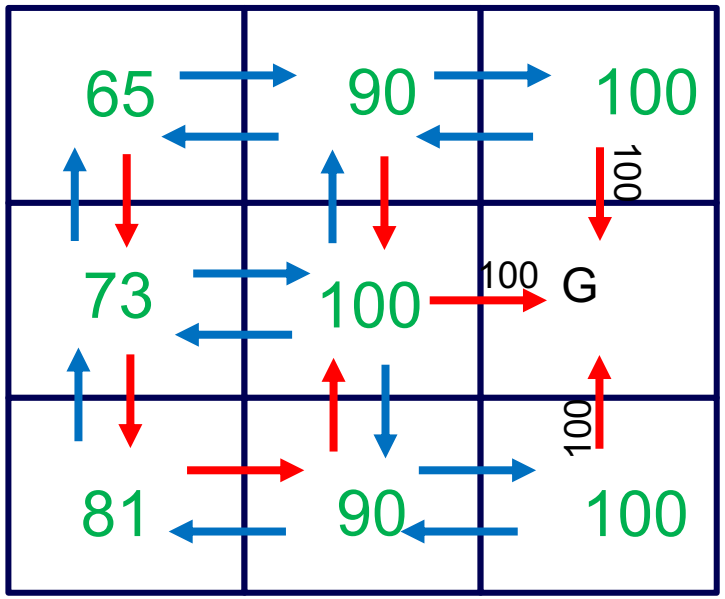
$\gamma = 0.9$

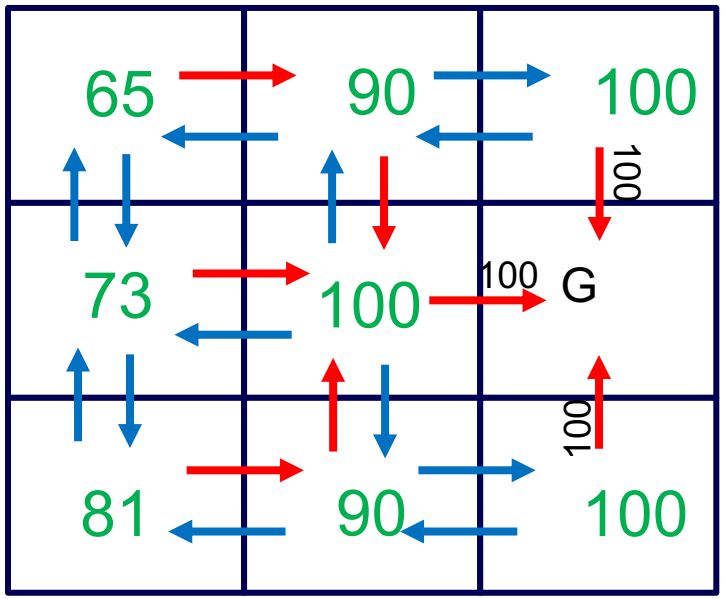
Policy Iteration



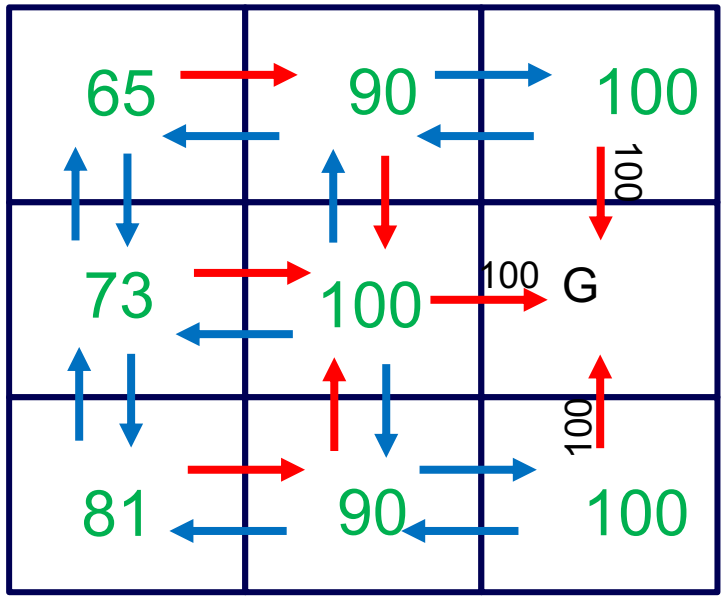


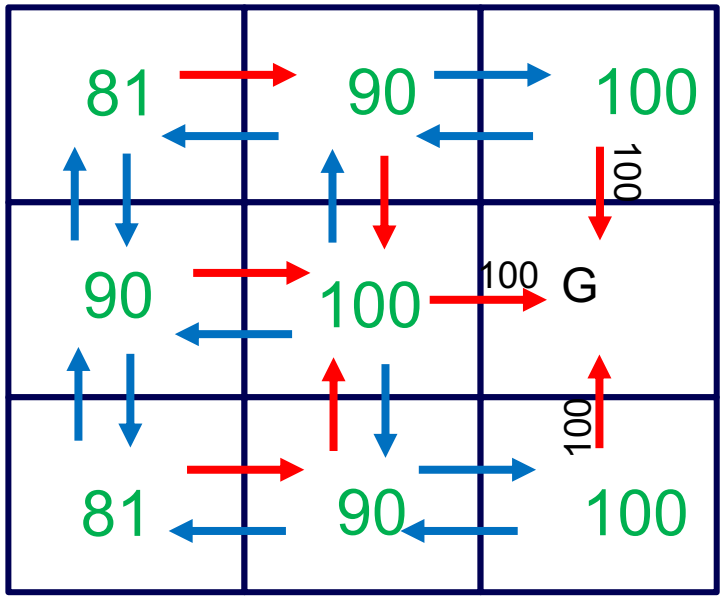
Policy Iteration



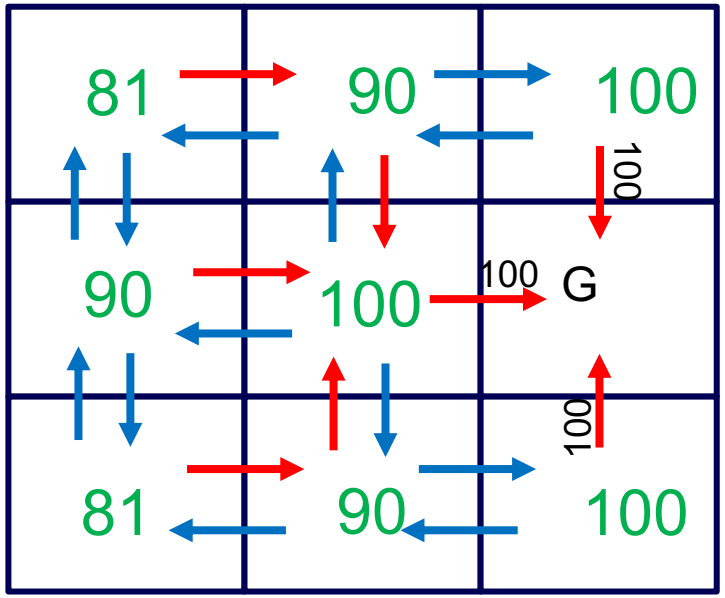


Policy Iteration

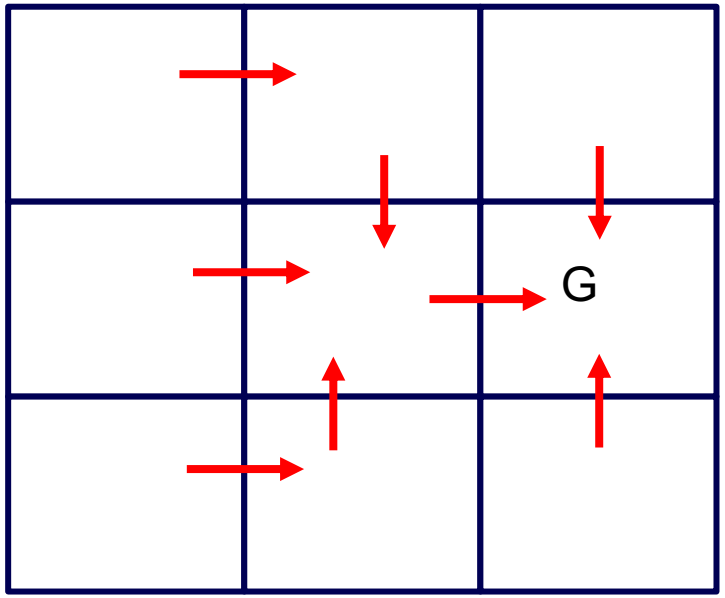




Policy Iteration



V^*



π^*

Value vs Policy Iteration

The two are very similar, and both converge to give the optimal policy

In general:

- Each main loop of value iteration is more efficient than policy iteration
- Policy iteration requires fewer main loops to converge than value iteration
- Policy iteration directly gives some a on each execution, and converges to an optimal policy on each iteration.
  - Provides an early termination option before convergence.

RMIT Classification: Trusted

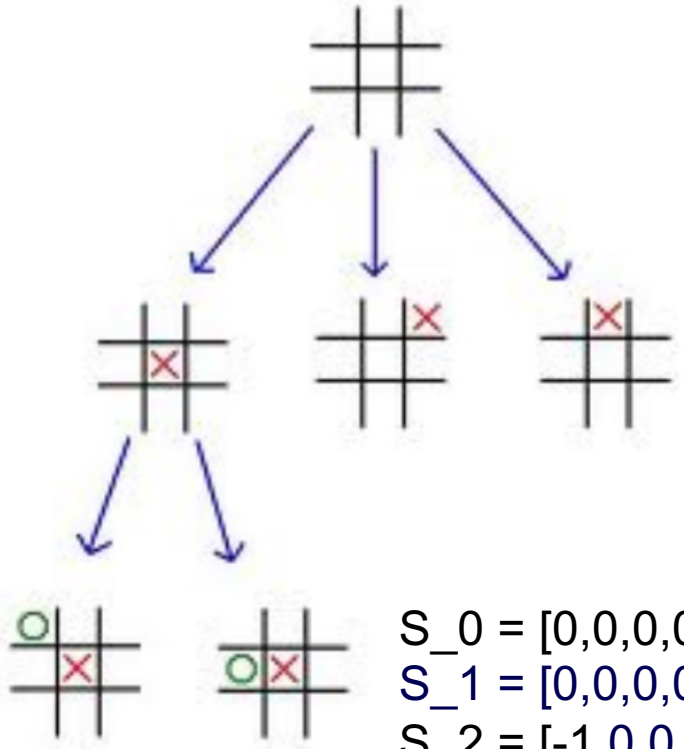
Model-Free RL

- Q-learning
- Temporal Difference Learning
- Exploration vs Exploitation

6 Things to Know

1. State Space, S
2. Action Set, A
3. ~~Transition Function, $T := S \times A \rightarrow S$~~
4. Reward Function, $R := S \times A \rightarrow \mathbb{R}$
5. Value Function, $V := S \rightarrow \mathbb{R}$
6. Policy Function, $\pi : S \rightarrow A$

Model-Free RL



S_0 = [0,0,0,0,0,0,0,0,0]
S_1 = [0,0,0,0,1,0,0,0,0]
S_2 = [-1,0,0,0,1,0,0,0,0]



$$V(s) = \max_a [R(s,a) + \gamma V(T(s,a))]$$

$$\pi(s) = \operatorname{argmax}_a [R(s,a) + \gamma V(T(s,a))]$$

Model-Free RL

For ***Model-Free RL***, the reward and transition functions are unknown

These functions must be discovered by the agent through trial-and-error

Additionally, without the reward or transition functions it is impossible to correctly update the value/policy functions:

$$V(s) = \max_a [R(s,a) + \gamma V(T(s,a))]$$
$$\pi(s) = \operatorname{argmax}_a [R(s,a) + \gamma V(T(s,a))]$$

- Both functions iterate over every possible state-action pair, which is impossible in a model-free setting

Q-Learning is one approach to Model-Free RL

Q-Learning

Q-Learning rephrases a RL problem in terms of the ***Q-function***

$$V^*(s) = E\left[R(s, \pi^*(s))\right] + \gamma E\left[V^*(s')\right]$$

$$Q(s, a) = R(s, a) + \gamma E\left[V^*(s')\right]$$

Similar to the value function, the Q-function measures the "worth" or "value" of a state-action pair. That is, what expected future reward of executing action a from state s

The value and policy functions can be rephrased to use the Q-function

Value function: $V(s) = \max_a Q(s, a)$

Policy function: $\pi(s) = \operatorname{argmax}_a Q(s, a)$

Q-Learning

The definition of the Q-function can be rewritten to be recursive

$$Q(s,a) = R(s,a) + \gamma E\left[V^*(s')\right]$$

$$Q(s,a) = R(s,a) + \gamma \max_{a'} Q\left(s', a'\right)$$

These definitions means that if some elements of the reward function and policy function are unknown, then a policy, though not necessarily optimal policy can be generated

	a_1	a_2	a_3
s_1	$Q(s_1, a_1)$	$Q(s_1, a_2)$	
s_2			
...			

Learning the Q-function

The RL problem has now been reduced to learning the Q-function:

- The agent explores the state-space, executing actions
- The agent observes the rewards
- The Q-function is updated
- The process continues until the Q-functions converge

Similar to the value/policy iteration, the Q-function is updated iteratively

- Initially, the Q-functions are unknown
- Each update is: $Q(s,a) = R(s,a) + \gamma \max_{a'} Q(s',a')$ where $s' = T(s,a)$

Learning the Q-function

Q-Learning($\langle S, A, T, R, V, \pi \rangle, \gamma$):

Forall $s \in S, a \in A$, initialise $Q(s, a)$ to small random value

Repeat until termination:

Observe the current state s

Choose action to execute a

Observe the new resulting state s'

Update the Q-function value:

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

Return policy function π

Exploration vs Exploitation

Q-Learning($\langle S, A, T, R, V, \pi \rangle, \gamma$):

Forall $s \in S, a \in A$, initialise $Q(s, a)$ to small random value

Repeat until termination:

Observe the current state s

Choose action to execute a

Observe the new resulting state s'

Update the Q-function value:

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

Exploration vs Exploitation

- **Exploration** finds more information about the environment
- **Exploitation** exploits known information to maximise reward

It is usually important to explore as well as exploit

Exploration vs Exploitation

Restaurant Selection

- Exploitation Go to your favourite restaurant
- Exploration Try a new restaurant

Online Banner Advertisements

- Exploitation Show the most successful advert
- Exploration Show a different advert

Oil Drilling

- Exploitation Drill at the best known location
- Exploration Drill at a new location

Game Playing

- Exploitation Play the move you believe is best
- Exploration Play an experimental move

Exploration vs Exploitation

If the state space is large or infinite, or there is limited training time:

- The agent cannot explore every state-action pair
- Instead, the agent should ***explore*** enough of the state-space to sufficient sample it
- Then the agent should ***exploit*** it's choices to optimise a small part of the Q-function

Exploration vs Exploitation

Introduce an: $\epsilon = [0,1]$

- ϵ indicates the proportion to which the agent should "learn" rather than rely on it's previous experience
- With probability ϵ, the agent choose a random action (**Explore**)
- With probability $1 - \epsilon$ the agent selects an action that follows it's current policy (**Exploit**)
- ϵ should start high and decrease over time (as the agent executes more actions)
- One definition for the learning rate is:
  - $\epsilon = \frac{1}{1+visits(s,a)}$

Probabilistic Transitions & Rewards

Probabilistic (or non-deterministic) RL problems result in:

- The result of a state-action pair (the result of a transition $T(s, a)$) is not always be the same
- The reward received for a state-action pair $R(s, a)$ is not always the same
- Both cause problems for updating the value/policy/Q-functions.

Consider the original Q-function

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

- This "forgets" any previous values that had been previously learnt about the state-action pair
- Thus, if the reward or transition function is probabilistic, the Q-function is unstable.

Temporal Difference Learning

Temporal Difference Learning balances the past and immediate experiences of the agent

- The agent cannot gain sufficient experience in a single instance/observation/trial
- Instead, experience is gathered temporally (over time)
- Initially, the agent should depend on it's immediate experience
- Over time, the agent should depend on its past experience

Temporal Difference Learning

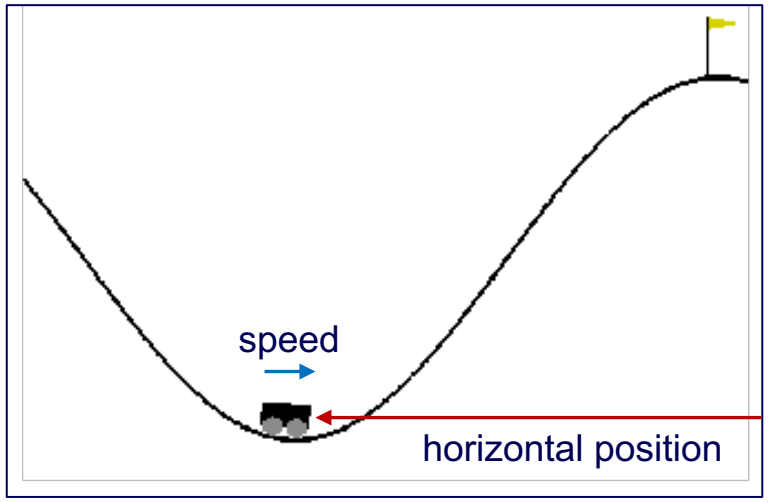
The Q-function update rule with TD-learning is:

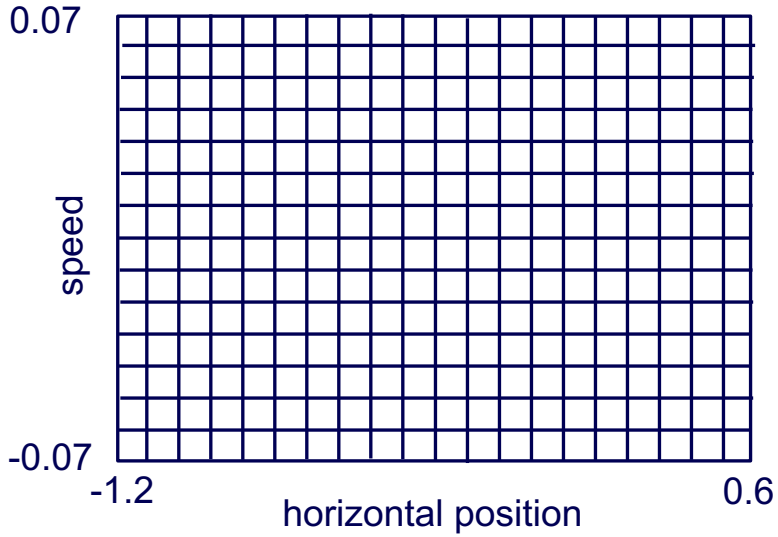
$$Q(s,a) = (1-\lambda)Q(s,a) + \lambda\left[R(s,a) + \gamma \max_{a'} Q(s',a')\right]$$

- This is the form of Q-learning that is generally presented
- The learning rate is used
- As λ decreases, the agent depends more on it's past experience
- In general, this converges to a stable value
- In practice, the learning rate is often a constant value: $\lambda = 0.1$

Demo

Mountain-Car with RL





1. State Space, S
2. Action Set, A
3. Transition Function, $T := S \times A \rightarrow A$
4. Reward Function, $R := S \times A \rightarrow \mathbb{R}$
5. Value Function, $V := S \rightarrow \mathbb{R}$
6. Policy Function, $\pi: S \rightarrow A$

$A \in \{$left (0), nothing (1), right (2)$\}$

Reward: -1 for every environment step until success

RMIT Classification: Trusted

Advanced RL

Advanced RL

There are a huge range of different approaches and modification to of Reinforcement Learning:

- Dynamic State-space decomposition, such as parti-graph
- Partially observable states (hidden variables)
- Semi-Markov problems with variable length actions
- Function approximation techniques
- Multiple goals
- Hierarchical Reinforcement learning
- Transfer learning with RL
- Deep Reinforcement Learning

All these variations share the same core of elements, and rely on the fundamental convergence results of the simple RL we've discussed.

Summary

- Reinforcement Learning
  - State Space
  - Action Set
  - Transition Function
  - Value Function
  - Reward Function
  - Policy
- Model-Based RL
  - Value Iteration
  - Policy Iteration
- Model-Free RL
  - Q-Learning
    - Temporal difference
  - Exploration vs Exploitation